PACKET CAPTURE: PACKET GENERATION

Byeong-hee Roh
Dept. of Software and Computer Engineering
Ajou University

변화와 융합의 중심
MMcN
멀티미디어 융합 연구실
Mobile Multimedia convergecne Network Lab.
http://mmcn.ajou.ac.kr

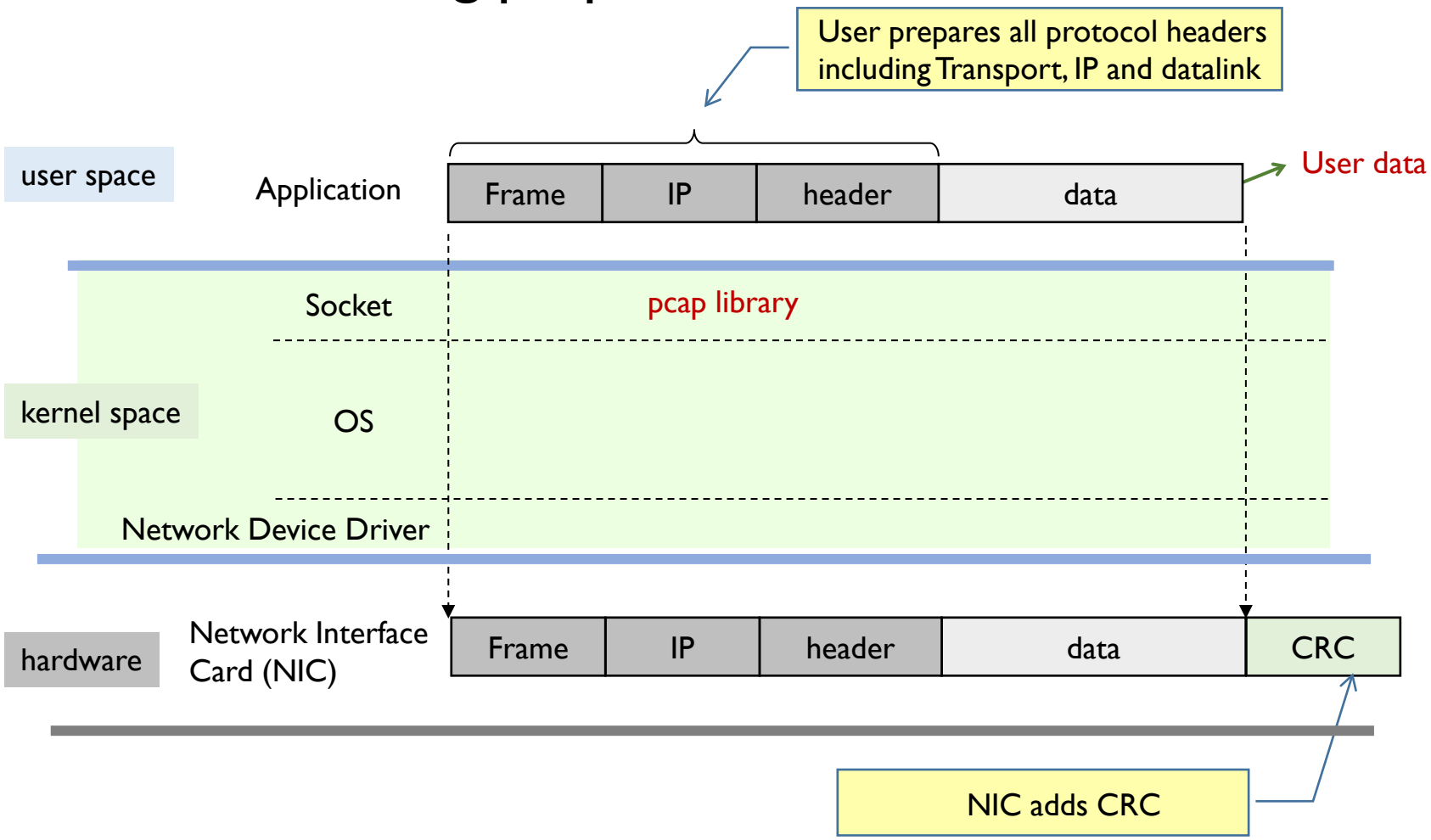
Contents

- Libpcap/WinPcap Packet Generation Functions
- libnet
- Linux PF_SOCKET
- Libpcap Packet Generation Example
- ARP Example

LIBPCAP/WINPCAP PACKET GENERATION FUNCTIONS

Packet Generation

❖ Packet Generation using pcap

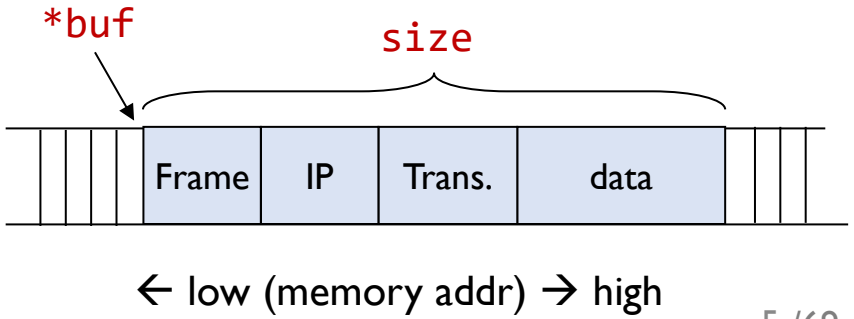


Packet Generation – pcap_inject( )

- pcap_inject( )
  - for libpcap
  - to sends a raw packet through the network interface

```
int   pcap_inject  ( pcap_t          *p,
                     const void      *buf,
                     size_t          size);
```

  - arguments
    - buf – pointer to the packet including the link-layer header
    - size – the number of bytes in the packet.
  - return value
    - the number of bytes written on success
    - -1 on failure



Packet Generation – pcap_inject( )

- Example

```
pcap_t *handle;

// Retrieve the device list
pcap_findalldevs (&alldevs, errbuf);

// Open the adapter
handle = pcap_open_live( d->name, 65536, 1, 1000, errbuf);

// Prepare a packet
const char packet[] = { /* packet data*/ };

int bytes_sent = pcap_inject(handle, packet, sizeof(packet));

if (bytes_sent == -1) {
    fprintf(stderr, "Error sending packet: %s\n", pcap_geterr(handle));
} else {
    printf("Sent %d bytes... success !!! \n", bytes_sent);
}
pcap_close(handle);
```

Packet Generation – pcap_sendpacket( )

- pcap_sendpacket( )
  - to sends a raw packet through the network interface

```
int   pcap_sendpacket  (   pcap_t          *p,
                           const u_char    *buf,
                           int             size);
```

  - arguments
    - buf – pointer to the packet including the link-layer header
    - size – the number of bytes in the packet.
  - return value
    - 0 on success
      - only the difference from pacp_inject( )
    - PCAP_ERROR (-1) on failure

Packet Generation – pcap_sendpacket( )

- Example

```
pcap_t *handle;

// Retrieve the device list
pcap_findalldevs (&alldevs, errbuf);

// Open the adapter
handle = pcap_open_live( d->name, 65536, 1, 1000, errbuf);

// Prepare a packet
const char packet[] = { /* packet data*/ };

int result = pcap_sendpacket(handle, (const u_char *)packet, sizeof(packet));
if (result != 0 ) {
    fprintf(stderr, "Error sending packet: %s\n", pcap_geterr(handle));
} else {
    printf("Packet sent success !!! \n");
}
pcap_close(handle);
```

Packet Generation – Compariosn

❖ pcap_inject vs. pcap_sendpacket( )

Feature	pcap_inject	pcap_sendpacket
Return Value	Number of bytes sent	Success/failure status (0/-1)
Packet Size Type	size_t	int
Success/Failure Check	Can indirectly verify through bytes sent	Directly indicates success or failure
Use Case	When the exact number of bytes sent needs to be verified	Used for simple packet transmission
Compatibility	Mostly used in standard libpcap environments like Linux	More common in Windows/WinPcap environments
Size limitation	No limitation to packet size upto size_t (but, depends on MTU, about 1500 bytes)	Since the packet size is passed as **int**, there may be restrictions when sending large packets (up to 2GB or more).

Packet Generation – Queue Management

- Other related functions
  - to allocate a send queue

```
pcap_send_queue* pcap_sendqueue_alloc ( u_int    memsize );
```

  - to add a packet at the end of the send queue

```
int pcap_sendqueue_queue ( pcap_send_queue*              queue,
                           const struct pcap_pkthdr*    pkt_header,
                           const u_char*                pkt_data);
```

  - to send a queue of raw packets to the network

```
u_int pcap_sendqueue_transmit (  pcap_t*                 p,
                                 pcap_send_queue*        queue,
                                 int                     sync);
```

  - to destroy a send queue

```
void pcap_sendqueue_destroy   (  pcap_send_queue*        queue );
```

Packet Generation – Example

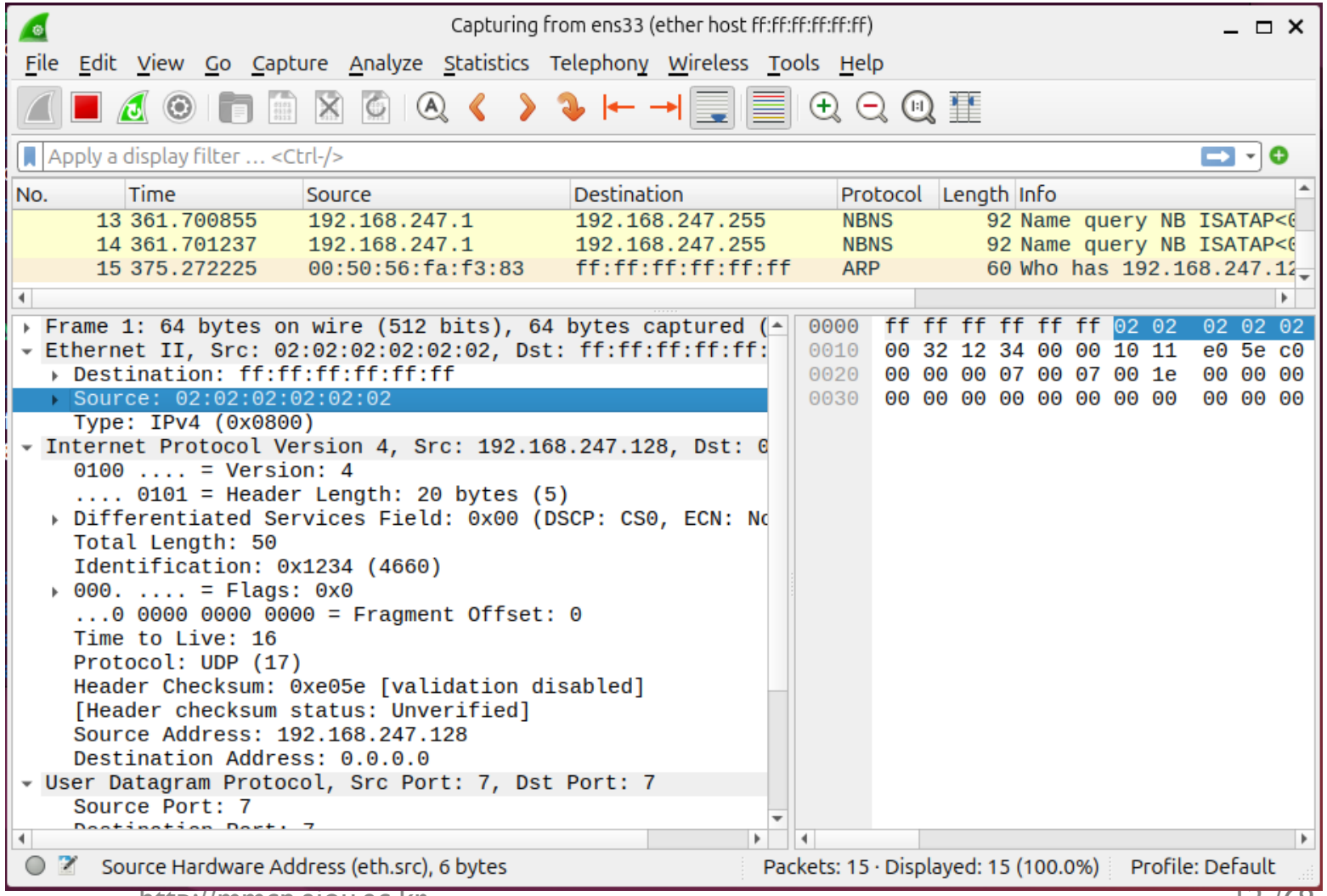
- npcap example – sendpack.c
  - in Linux, change the line 131 as

```
#ifdef _WIN32
  *(u_long *)(packet + 14 + 12) = ((struct sockaddr_in *)(addr->addr))->sin_addr.S_un.S_addr;
#else
  *(u_long *)(packet + 14 + 12) = ((struct sockaddr_in *)(addr->addr))->sin_addr.s_addr;;
#endif
```

  - usage
    - `printf("usage: %s <interface>", argv[0]);`
- Prctice
  - add routines to select interface from the list of all device interfaces

Packet Generation – Example

```
$ gcc sendpack.c -lpcap
$ sudo ./a.out ens33
```



LIBNET

libnet

- libnet
  - https://github.com/libnet/libnet
  - a C library that simplifies
    - the creation, manipulation, and transmission of network packets
  - primarily used for low-level network programming
    - making it easier for tasks like penetration testing, network simulation, and custom protocol creation
    - providing a high-level API to construct packets for various network protocols,
      - such as TCP, UDP, ICMP, ARP, user-customized protocols, and more
      - with each control over packet structure and content
      - with handles the complexities of packet crafting and allows easy control over packet structure and content.

libnet

- Benefit of libnet
  - complex network packet generation works
    - can be handled with simple function calls
  - easily combine multiple protocols
    - to create multi-protocol packets
  - can be used for various purposes
    - such as network analysis, testing, and attack simulation
- Limitation of libnet
  - Due to the nature of low-level programming
    - it may be difficult for beginners to use
  - There may be platform-specific dependencies
    - Some features may be limited for newer network protocols
  - It is ideally targeted for packet generation and transmission
    - not providing packet capture

Install libnet

- For linux

```
sudo apt-get install libnet-dev
```

- For macOS

```
brew install libnet
```

- For Windows
  - It can write program with Npcap utilizing libnet source code

libnet programming

- basic programming procedure using libnet (1/2)
  - initialize libnet

```
l = libnet_init(LIBNET_RAW4, NULL, errbuf);
```

  - create a packet (e.g. TCP)

```
u_long dest_ip = libnet_name2addr4(l, "192.168.1.1", LIBNET_RESOLVE);
// Build TCP packet
libnet_build_tcp(
        12345,                  // Source Port
        80,                     // Destination Port
        0x10000000,             // Sequence Number
        0x20000000,             // Acknowledgment Number
        TH_SYN,                 // Flags (SYN)
        32767,                  // Window Size
        0,                      // Checksum (0 for auto-calculation)
        0,                      // Urgent Pointer
        LIBNET_TCP_H,           // Total Length
        NULL,                   // Payload
        0,                      // Payload Size
        l,                      // Libnet context
        0                       // Protocol Tag
);
```

libnet programming

- basic programming procedure using libnet (2/2)
  - send packet

```
res= libnet_write(l)
if ( res == -1)
   fprintf(stderr, "libnet_write() failed: %s\n", libnet_geterror(l));
```

  - clean up

```
libnet_destroy(l);
```

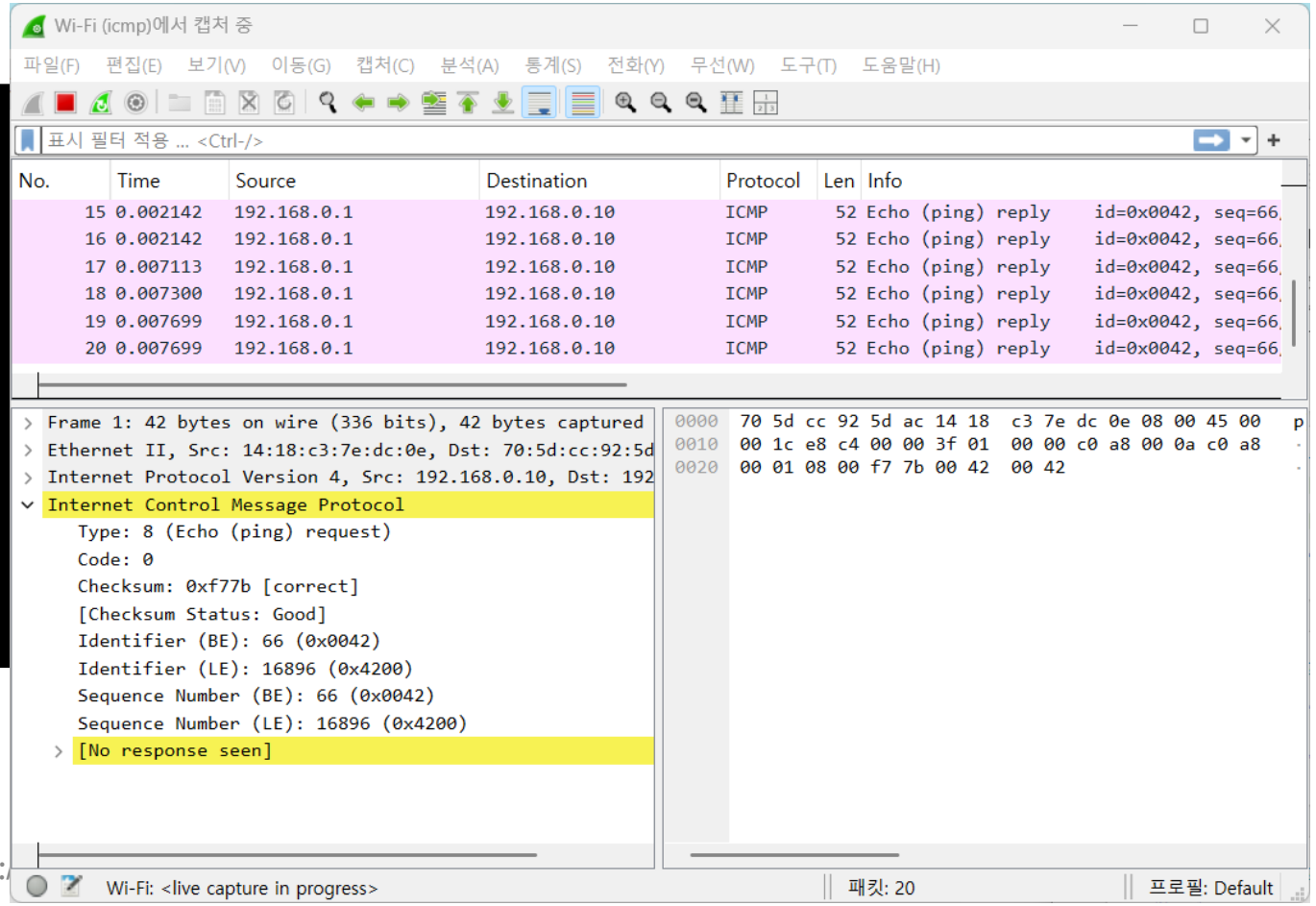
  - compile

```
gcc <program file> -lnet
```

Sample

- icmp_echo_cq.c
  - https://github.com/libnet/libnet/blob/master/sample/icmp_echo_cq.c

```
$ gcc icmp_echo_cq.c -lnet
$ sudo ./a.out -s 192.168.0.10 -d 192.168.0.1
libnet 1.1 packet shaping: ICMP[RAW using context queue]
Wrote 28 byte ICMP packet from context "echo 9"; check the wire.
Wrote 28 byte ICMP packet from context "echo 8"; check the wire.
Wrote 28 byte ICMP packet from context "echo 7"; check the wire.
Wrote 28 byte ICMP packet from context "echo 6"; check the wire.
Wrote 28 byte ICMP packet from context "echo 5"; check the wire.
Wrote 28 byte ICMP packet from context "echo 4"; check the wire.
Wrote 28 byte ICMP packet from context "echo 3"; check the wire.
Wrote 28 byte ICMP packet from context "echo 2"; check the wire.
Wrote 28 byte ICMP packet from context "echo 1"; check the wire.
Wrote 28 byte ICMP packet from context "echo 0"; check the wire.
```



LINUX PF_SOCKET

PF_PACKET

- PF_PACKET
  - a socket type used on Linux for packet capture
    - allows to access raw packets directly from the network interface
  - libpcap uses PF_PACKET also, providing
    - a high-level library that simplifies packet capturing
    - a cleaner API to interact with various network interfaces

PF_PACKET

❖ Usage procedure (1/2)

- socket creation

```
int sock = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
```

- interface binding

```
struct ifreq ifr;
struct sockaddr_ll sll;

strncpy(ifr.ifr_name, "eth0", IFNAMSIZ); // set interface name
res = ioctl(sock, SIOCGIFINDEX, &ifr) ;

memset(&sll, 0, sizeof(sll));
sll.sll_family   = AF_PACKET;
sll.sll_ifindex  = ifr.ifr_ifindex; // 인터페이스 인덱스
sll.sll_protocol = htons(ETH_P_ALL);

bind(sock, (struct sockaddr *)&sll, sizeof(sll))
```

PF_PACKET

❖ Usage procedure (2/2)

- packet generation

```
unsigned char *packet;

int len = sendto(sock, packet, sizeof(packet), 0, (struct sockaddr *)&sll, sizeof(sll));
```

- packet reception

```
unsigned char buffer[2048];

int len = recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);
```

PF_PACKET

❖ Example (1/3)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <net/if.h>
#include <linux/if_packet.h>
#include <netinet/ether.h>
#include <arpa/inet.h>
#include <sys/ioctl.h>

int main() {
    int sock;
    struct ifreq ifr;
    struct sockaddr_ll sll;
    unsigned char buffer[2048];
```

```
// 소켓 생성
sock = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
if (sock == -1) {
    perror("Socket creation failed");
    return -1;
}

// 네트워크 인터페이스 가져오기
strncpy(ifr.ifr_name, "ens33", IFNAMSIZ);
if (ioctl(sock, SIOCGIFINDEX, &ifr) == -1) {
    perror("Failed to get interface index");
    close(sock);
    return -1;
}

// 소켓 주소 설정
memset(&sll, 0, sizeof(sll));
sll.sll_family = AF_PACKET;
sll.sll_ifindex = ifr.ifr_ifindex;
sll.sll_protocol = htons(ETH_P_ALL);
```

PF_PACKET

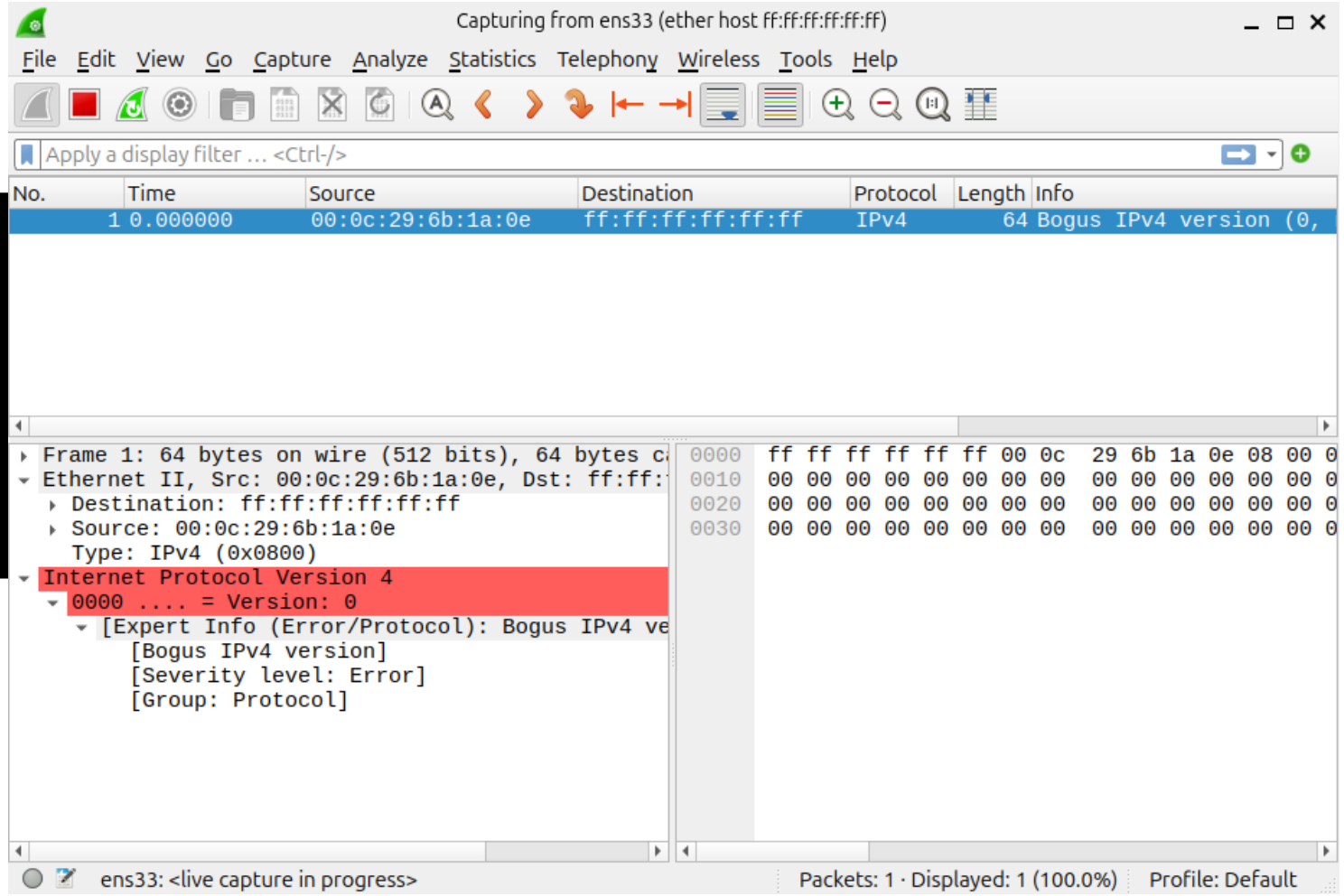
❖ Example (2/3)

```
    if (bind(sock, (struct sockaddr *)&sll, sizeof(sll)) == -1) {
        perror("Bind failed");
        close(sock);
        return -1;
    }

    unsigned char packet[64] = {
       0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,  // 목적지 MAC (브로드캐스트)
       0x00, 0x0C, 0x29, 0x6B, 0x1A, 0x0E,  // 출발지 MAC
       0x08, 0x00,                          // 이더넷 타입 (IPv4)
    };

    // 패킷 수신
    int len = sendto(sock, packet, sizeof(packet), 0, (struct sockaddr *)&sll, sizeof(sll));
    printf("Packet sent successfully\n");
                                    }

    // 패킷 수신
    printf("Listening for packets...\n");
    int len = recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);
    printf("Received packet of length %d\n", len);
    close(sock);
    return 0;
}
```

PF_PACKET

- Example (3/3)

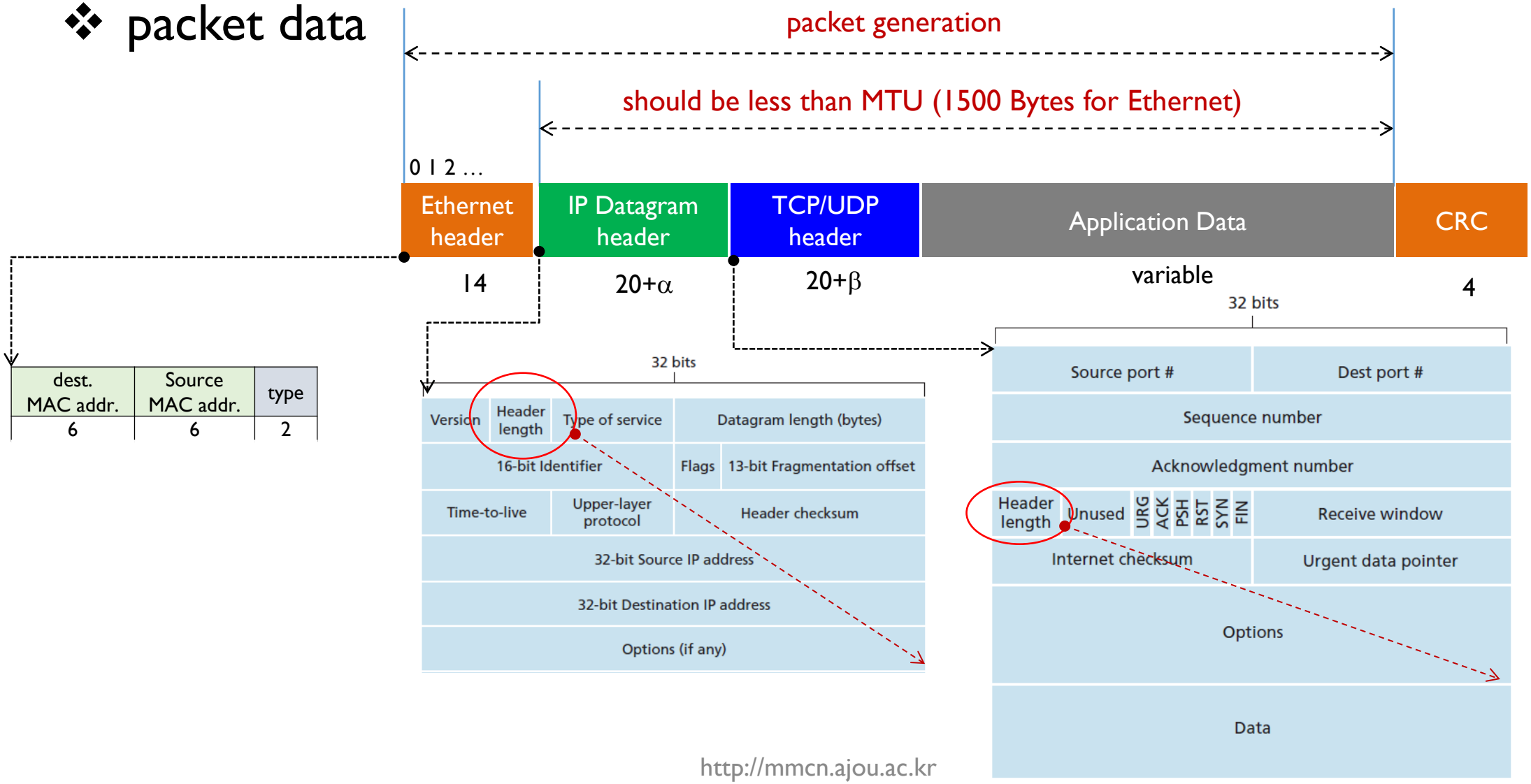
```
$ gcc pf_ex.c
$ sudo ./a.out
Packet sent successfully
Listening for packets...
```



Libpcap Packet Generation Example

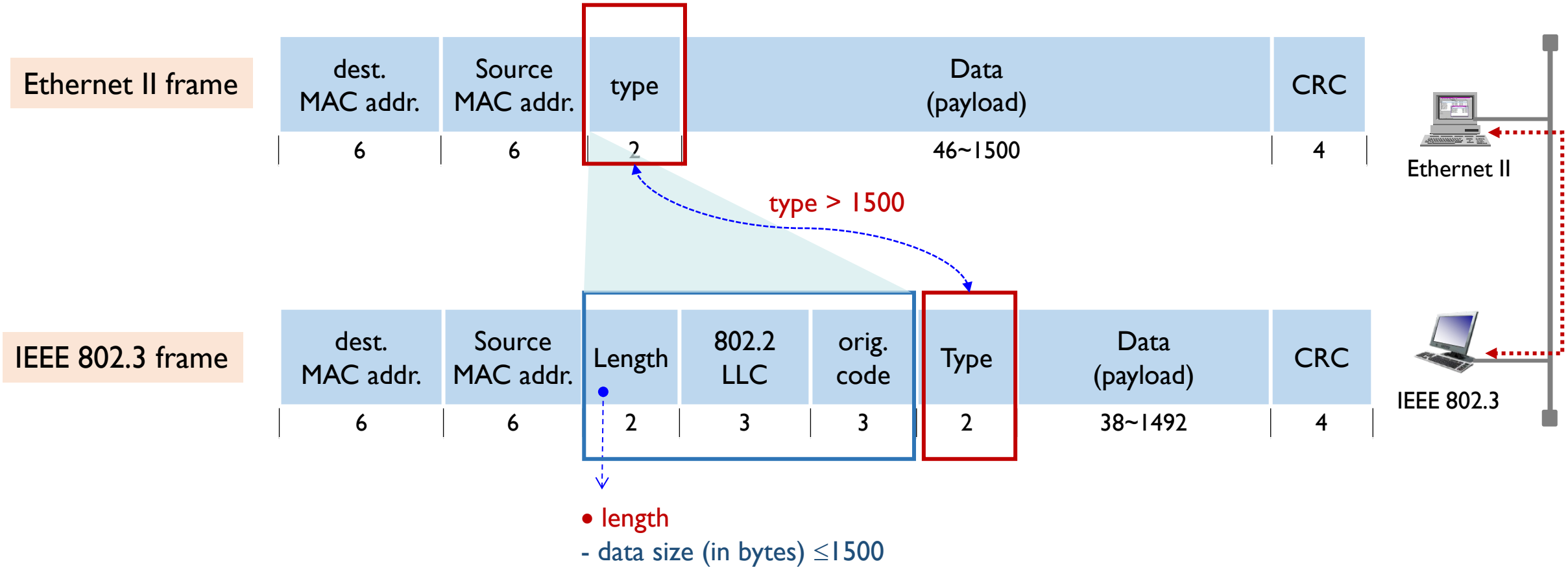
Packet Generation Data Structure

❖ packet data



Ethernet Header Generation (1/3)

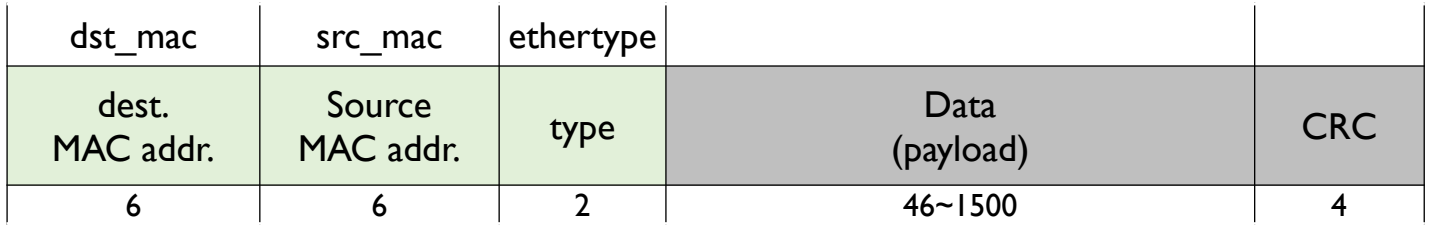
❖ Ethernet II and IEEE 802.3 Ethernet frame structure



Ethernet Header Generation (2/3)

- Ethernet header data structure
  - by referring to /usr/include/linux/if_ether.h in linux

```
#define         MAC_ADDR_LEN    6       // length of MAC address (in bytes)
#define         ETHERTYPE_IP    0x0800  // IP protocol
#define         ETHERTYPE_ARP   0x0806  // ARP protocol

// Ethernet Header Structure
struct eth_header {
    uint8_t  dst_mac[6];  // Destination MAC Address
    uint8_t  src_mac[6];  // Source MAC Address
    uint16_t ethertype;   // EtherType (e.g., IPv4: 0x0800)
};
```



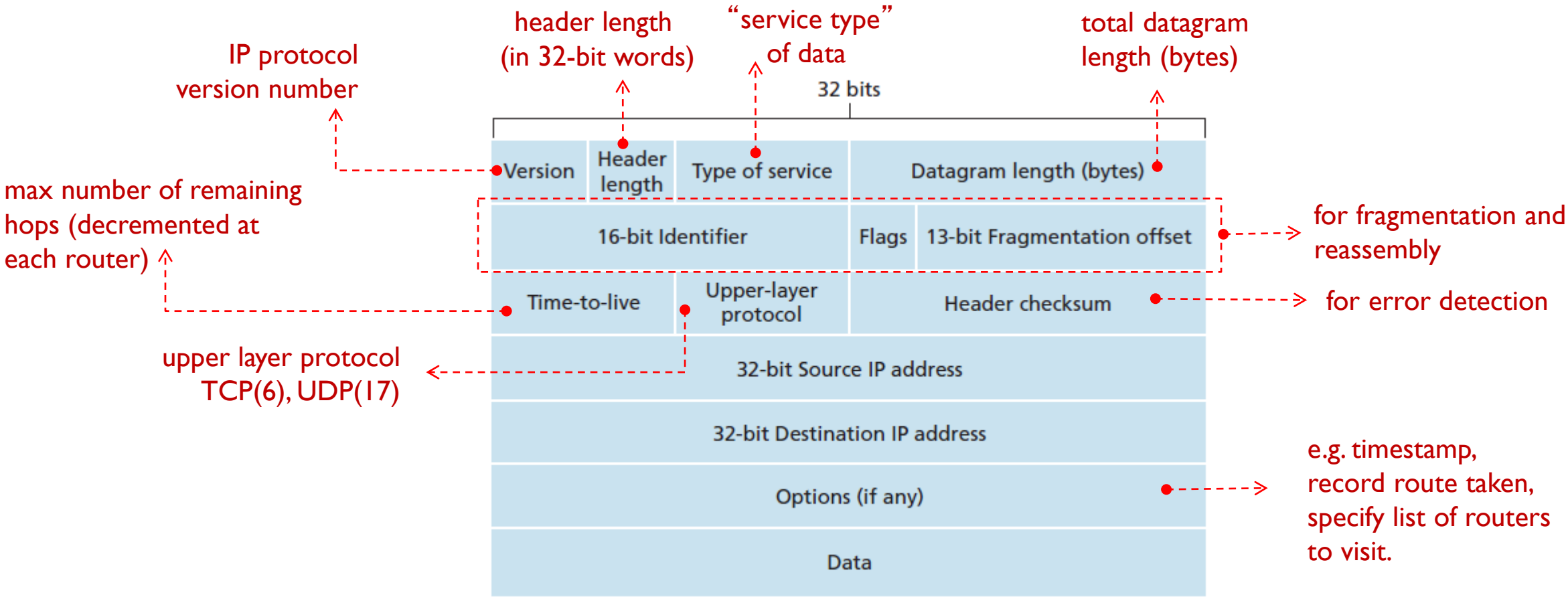
Ethernet Header Generation (3/3)

- Example code to generate Ethernet header

```
// Ethernet header fields example
uint8_t dst_mac[6] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };   // Broadcast MAC
uint8_t src_mac[6] = { 0x0a, 0x0b, 0x0c, 0x01, 0x02, 0x03 };   // Source MAC (temp.)
uint16_tetype = 0x0800;

// Ethernet header generation
void get_header_ethernet(struct eth_header *eth) {

    // copy destination mac to ethernet header
    memcpy(eth->dst_mac, dst_mac, 6);

    // copy source mac to ethernet header
    memcpy(eth->src_mac, src_mac, 6);

    // EtherType (network byte order)
    eth->ethertype = htons(etype);
}
```

IP Header Generation (1/4)

❖ IP datagram header format



IP Header Generation (2/4)

- IP datagram header data structure
  - by referring to /usr/include/linux/ip.h in linux

```
// IP header sturcute (IPv4)
struct ip_header {
    uint8_t  ihl : 4;                   // Header Length
    uint8_t  version : 4;               // IP Version
    uint8_t  tos;                       // Type of Service
    uint16_t total_length;              // Total Length
    uint16_t id;                        // Identification
    uint16_t fragment_offset;           // Fragment Offset
    uint8_t  ttl;                       // Time to Live
    uint8_t  protocol;                  // Protocol (TCP: 6, UDP: 17)
    uint16_t checksum;                  // Header Checksum
    uint32_t src_ip;                    // Source IP Address
    uint32_t dst_ip;                    // Destination IP Address
};
```

IP Header Generation (3/4)

❖ Example code to generate IP header (1/2)

```
// IP hdaerd fields example
char    *sip = "192.168.0.110";
char    *dip = "192.168.0.120";
uint8_t protocol = IPPROTO_UDP;

void get_header_ip(struct ip_header* iph)
{
    uint32_t            src_ip, dst_ip;
    uint16_t            upp_len, checksum = 0;
    uint8_t             buffer[4096] = { 0 };

    // IP header fields example
    src_ip = inet_addr(sip);                    // source IP address
    dst_ip = inet_addr(dip);                    // destination IP address

    if (protocol == IPPROTO_TCP)
        upp_len = 20 + sizeof(struct tcp_header) + strlen(app_data);
    else
        upp_len = 20 + sizeof(struct udp_header) + strlen(app_data);
```

IP Header Generation (4/4)

❖ Example code to generate IP header (2/2)

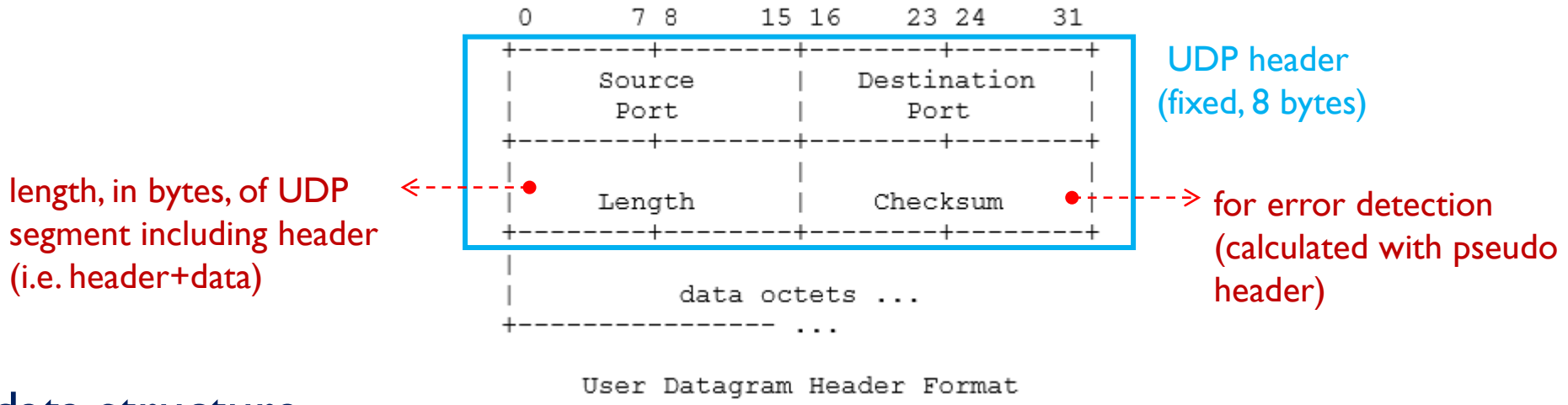
```
    iph->version        = 4;            // IPv4
    iph->ihl            = 5;            // Header Length (5 words), no option
    iph->tos            = 0;            // Default TOS
    iph->total_length   = htons(upp_len);       // no option
    iph->id             = htons(54321);         // Example Identification
    iph->fragment_offset = 0;           // No fragmentation
    iph->ttl            = 64;           // Default TTL
    iph->protocol       = protocol;     // Protocol (TCP, UDP, ICMP, etc)
    iph->checksum       = 0;            // Will be calculated later
    iph->src_ip         = src_ip;       // Convert to network byte order
    iph->dst_ip         = dst_ip;       // Convert to network byte order

    memcpy(buffer, iph, sizeof(struct ip_header));

    // checksum calculation
    iph->checksum       = calculate_checksum(buffer, sizeof(struct ip_header));
}
```

Checksum Calculation

```
// Checksum Calculation
uint16_t calculate_checksum(uint8_t* buffer, int length) {
    uint32_t    checksum= 0;
    uint16_t*   buffer_16       = (uint16_t*)buffer;

    // Padding zero for odd number buffer
    if (length % 2 == 1) {
        buffer[length] = 0;
        length += 1;
    }

    while (length > 1) {
        checksum += *buffer_16++;  // 2bytes
        length -= 2;
    }
    // Wrap around for carry over 16-bits length
    checksum = (checksum >> 16) + (checksum & 0xFFFF);

    // one's complement
    return ~checksum;
}
```

UDP Header Generation (1/5)

❖ UDP header format



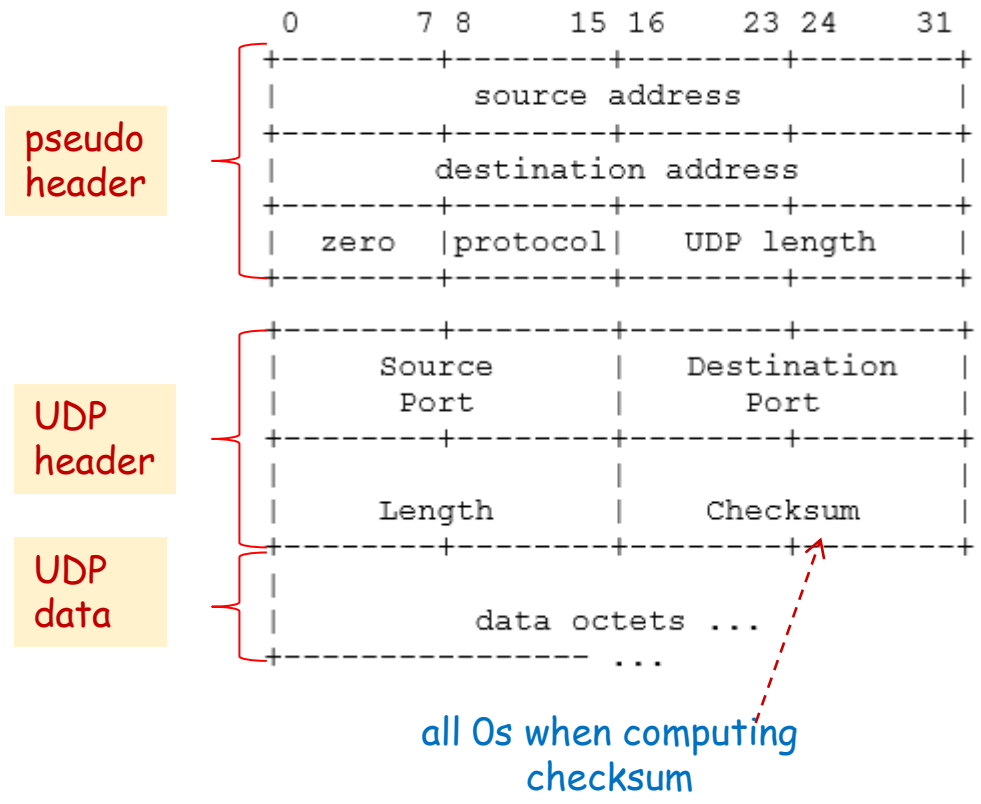
User Datagram Header Format

- data structure

```
// UDP header sturcute
struct udp_header {
    uint16_t src_port;          // Source Port
    uint16_t dst_port;          // Destination Port
    uint16_t length;            // UDP Length
    uint16_t checksum;          // UDP Checksum
};
```

UDP Header Generation (2/5)

- UDP pseudo header for checksum calculation




```
// Pseudo-header for checksum calculation
struct pseudo_header {
    uint32_t  src_addr;  // Source IP Address
    uint32_t  dst_addr;  // Destination IP Address
    uint8_t   zeros;     // zero bits
    uint8_t   protocol;  // protocol field in IP header
    uint16_t  length;    // length of header + data
};
```

UDP Header Generation (3/5)

❖ example code to generate UDP header (1/2)

```
// UDP header fields example
uint16_t    src_port    = 9999;
uint16_t    dst_port    = 80;

// Application data
char        *app_data = "Hello World, I am the Hero !!!";

void   get_header_udp(struct udp_header* udph)
{
    struct pseudo_header     psh;
    uint32_t                 checksum = 0;
    uint8_t                  buffer[4096] = { 0 };
    int                      hlen;

    // UDP header fields without checksum
    udph->src_port    = htons(src_port);    // Convert to network byte order
    udph->dst_port    = htons(dst_port);    // Convert to network byte order
    udph->length      = htons(sizeof(struct udp_header) + strlen(app_data));
    udph->checksum    = 0;                  // Optional for UDP, can be left as 0
```

UDP Header Generation (4/5)

❖ example code to generate UDP header (2/2)

```
    // Get pseudo header
    get_header_pseudo(&psh);

    // Concatenate packets Pseudo, UDP and App data into one
    memcpy(buffer, &psh, sizeof(struct pseudo_header));
    memcpy(buffer + sizeof(struct pseudo_header),
              udph, sizeof(struct udp_header));
    memcpy(buffer + sizeof(struct pseudo_header) + sizeof(struct udp_header),
              app_data, strlen(app_data));

    // UDP checksum calculation
    hlen = (int)sizeof(struct pseudo_header) + (int)sizeof(struct udp_header)
              + (int)strlen(app_data);

    udph->checksum = calculate_checksum(buffer, hlen);

}
```

UDP Header Generation (5/5)

- example code to generate Pseudo header

```
voidget_header_pseudo(struct pseudo_header *psh)
{
    uint32_t                src_ip, dst_ip;

    // IP header fields example
    src_ip = inet_addr(sip);                // source IP address
    dst_ip = inet_addr(dip);                // destination IP address

    // pseudo header
    psh->src_addr     = src_ip;
    psh->dst_addr     = dst_ip;
    psh->zeros        = 0;
    psh->protocol     = protocol;
    if (protocol == IPPROTO_TCP)
       psh->length = htons(sizeof(struct tcp_header) + strlen(app_data));
    else
       psh->length = htons(sizeof(struct udp_header) + strlen(app_data));
}
```

UDP Packet Generation – All Together (1/2)

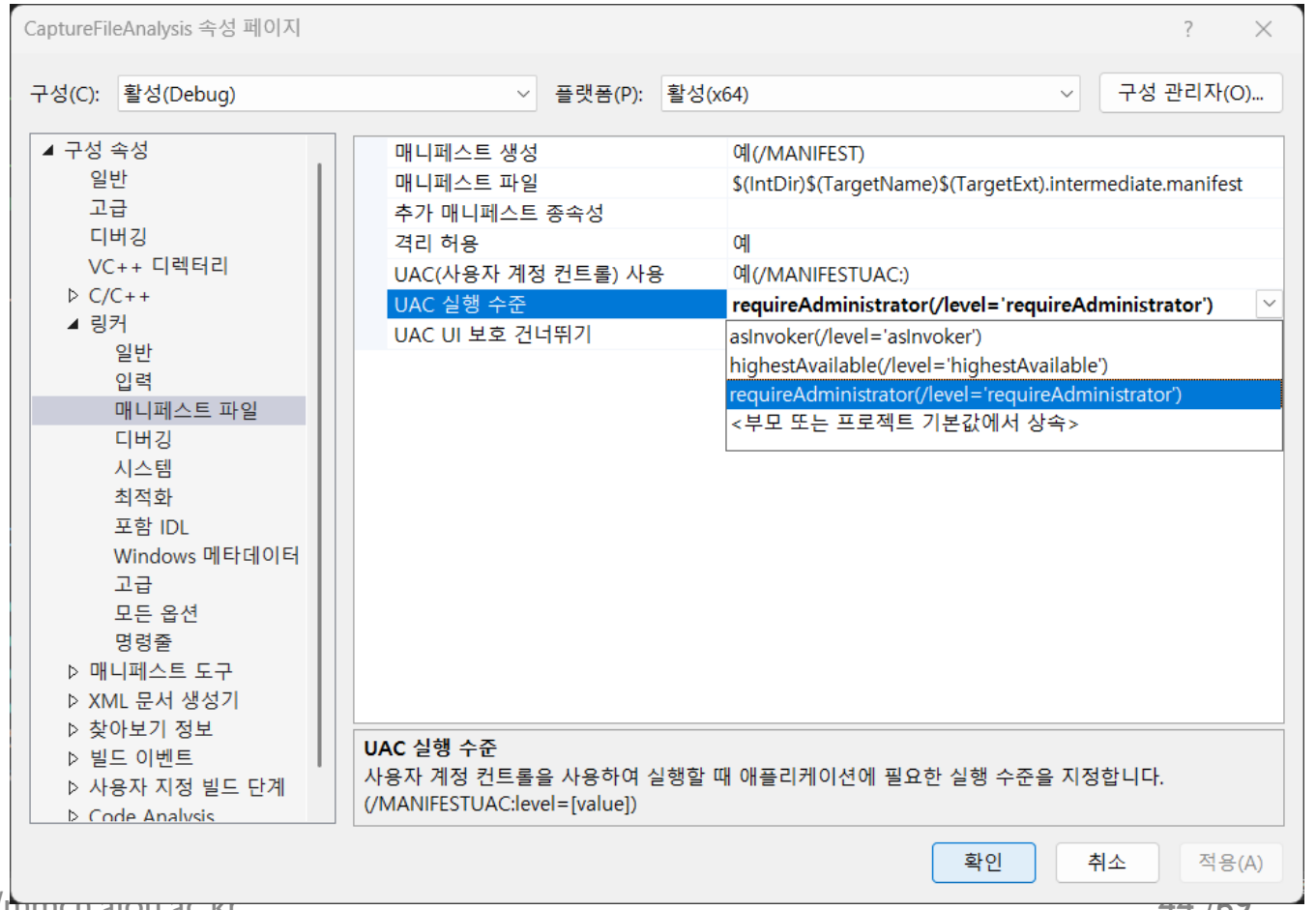
```
void   generate_packet_tcp_udp(u_char* packet, int *pklen) {
    char                    sendbuf[4096] = { 0 };
    struct eth_header       *eth;
    struct ip_header        *iph;
    struct udp_header       *udph;
    char                    *udata;

    // Concatenate Ethernet, IP, UDP and Data into sendbuf
    eth       = (struct eth_header*)sendbuf;
    iph       = (struct ip_header*)(sendbuf + sizeof(struct eth_header));
    udph      = (struct udp_header*)(sendbuf + sizeof(struct eth_header)
                    + sizeof(struct ip_header));
    udata     = (char*)(sendbuf + sizeof(struct eth_header)
                    + sizeof(struct ip_header) + sizeof(struct udp_header));
    *pklen    = (int)sizeof(struct eth_header) + (int)sizeof(struct ip_header)
                    + (int)sizeof(struct udp_header) + (int)strlen(udata);

    strcpy(udata, app_data);        // Application data
    get_header_udp(udph);           // UDP header
    get_header_ip(iph);             // ip header
    get_header_ethernet(eth);       // Ethernet header
}
```

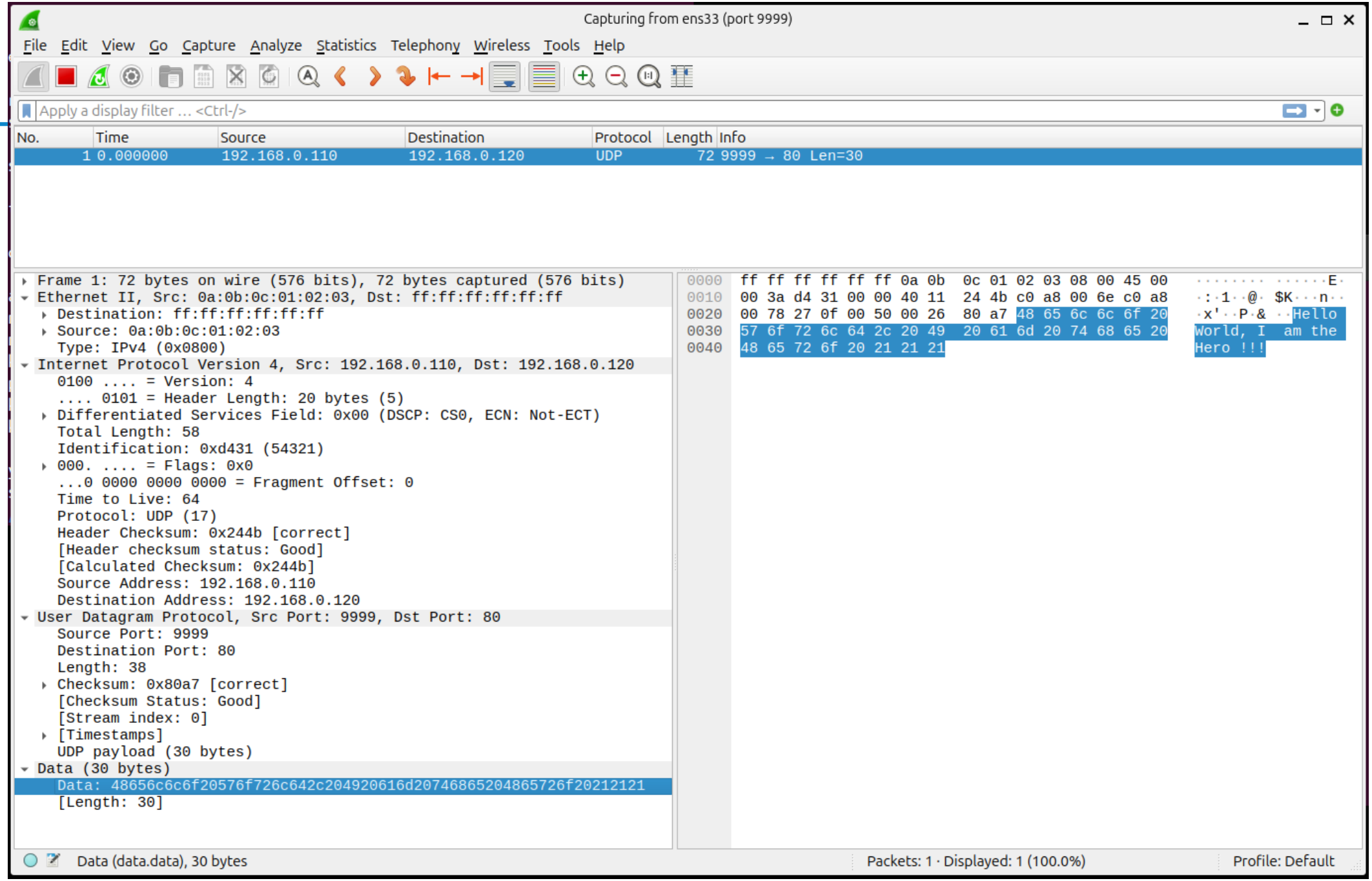
UDP Packet Generation – All Together (2/2)

```
int main()
{
    pcap_if_t               *alldevs;
    pcap_if_t               *d;
    int                     i, dnum, snum;

    pcap_findalldevs(&alldevs, errbuf);     // Retrieve the device list

    ifprint0(alldevs, &dnum);               // Scan the list printing every entry

    // Select the interface for handling packets
    printf("Enter the interface number (1-%d):", dnum);
    scanf("%d", &snum);

    for (d = alldevs, i = 0; i < snum - 1; d = d->next, i++);

    adhandle = pcap_open_live(d->name, 65536, 1, 1000, effbuf);   // Open the adapter

    generate_packet_tcp_udp(packet, &pklen);                      // UDP segment generation

    pcap_sendpacket(adhandle, packet, pklen);      // Send packet
}
```

Tips

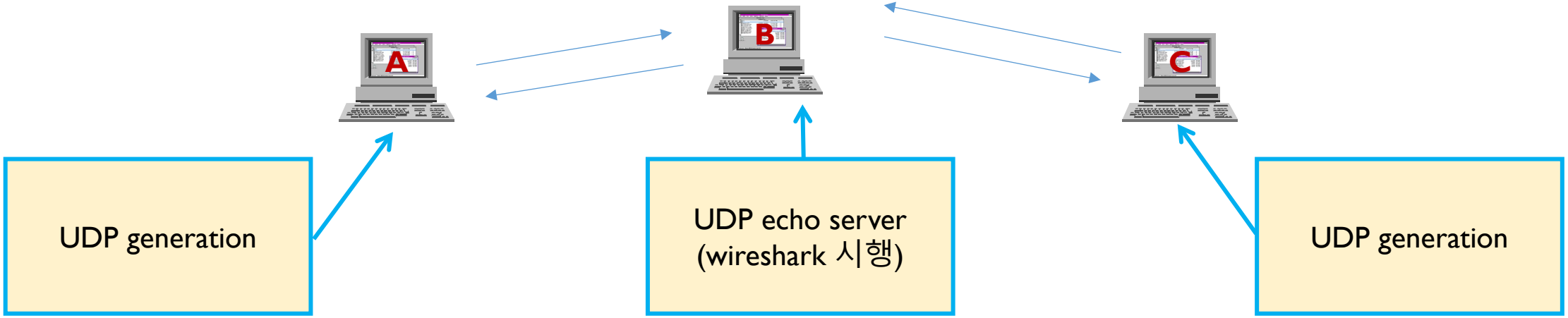
- For Windows
  - to run debug-level execution, set UAC level needs to be Admin. Privilege
- For Linux or macOS
  - run as sudo…




Capturing from ens33 (port 9999)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000 192.168.0.110 192.168.0.120 UDP 72 9999 → 80 Len=30
Frame 1: 72 bytes on wire (576 bits), 72 bytes captured (576 bits)
Ethernet II, Src: 0a:0b:0c:01:02:03, Dst: ff:ff:ff:ff:ff:ff
Destination: ff:ff:ff:ff:ff:ff
Source: 0a:0b:0c:01:02:03
Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.0.110, Dst: 192.168.0.120
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 58
Identification: 0xd431 (54321)
000. .... = Flags: 0x0
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: UDP (17)
Header Checksum: 0x244b [correct]
[Header checksum status: Good]
[Calculated Checksum: 0x244b]
Source Address: 192.168.0.110
Destination Address: 192.168.0.120
User Datagram Protocol, Src Port: 9999, Dst Port: 80
Source Port: 9999
Destination Port: 80
Length: 38
Checksum: 0x80a7 [correct]
[Checksum Status: Good]
[Stream index: 0]
[Timestamps]
UDP payload (30 bytes)
Data (30 bytes)
Data: 48656c6c6f20576f726c642c204920616d20746865204865726f20212121
[Length: 30]
0000 ff ff ff ff ff ff 0a 0b 0c 01 02 03 08 00 45 00
0010 00 3a d4 31 00 00 40 11 24 4b c0 a8 00 6e c0 a8
0020 00 78 27 0f 00 50 00 26 80 a7 48 65 6c 6c 6f 20
0030 57 6f 72 6c 64 2c 20 49 20 61 6d 20 74 68 65 20
0040 48 65 72 6f 20 21 21 21
Hello World, I am the Hero !!!
Data (data.data), 30 bytes
Packets: 1 · Displayed: 1 (100.0%)
Profile: Default

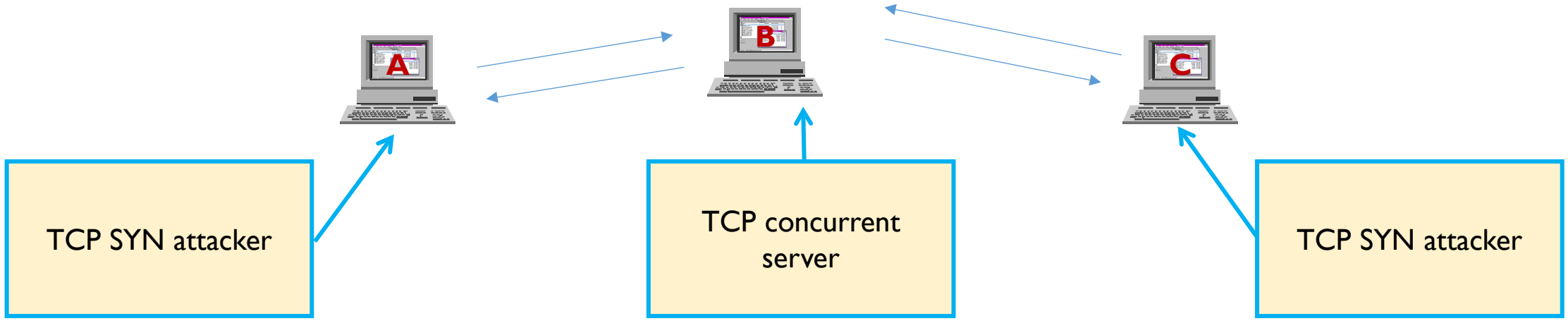
Practice #1

❖ UDP echo client server



Practice #2

- TCP SYN Flooding
  - UDP generation program을 활용하여 TCP generation program을 작성
  - SYN segement를 연속으로 서버에 전송

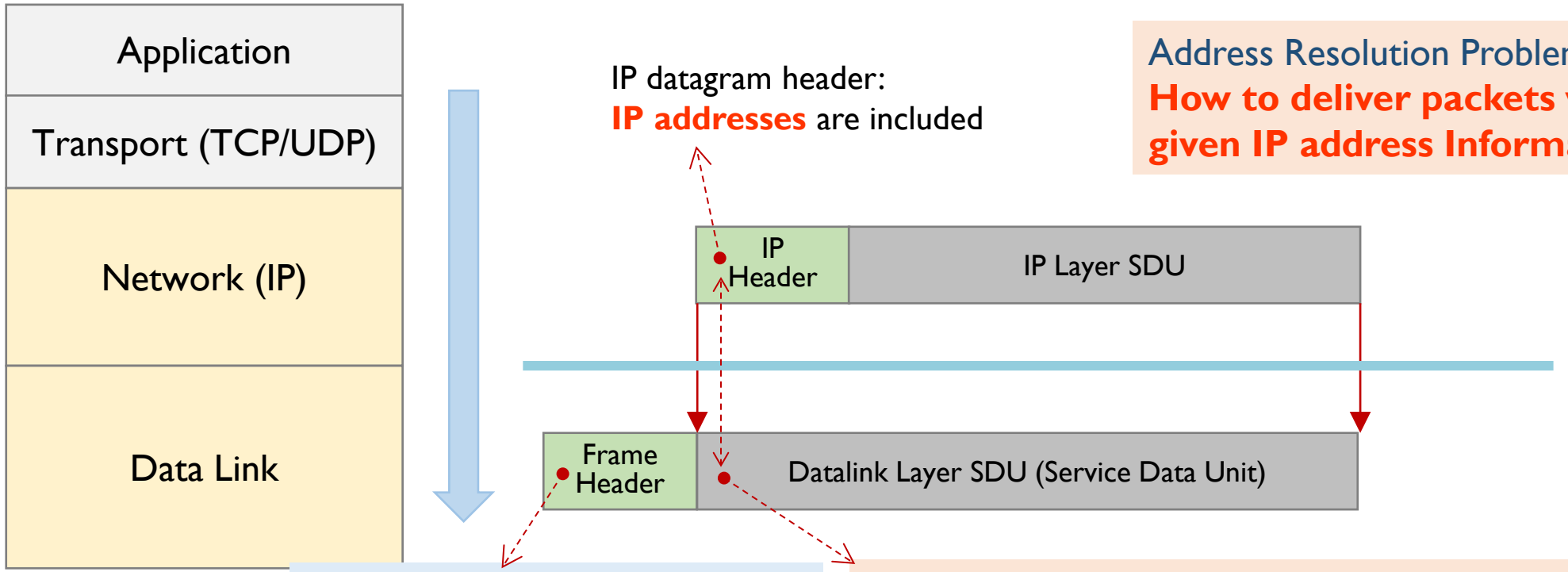


ARP Example

ARP (Address Resolution Protocol)

❖ Address Resolution Problem

- IP address at layer 3 is encapsulated (not seen) at datalink layer



Address Resolution Problem:
How to deliver packets with given IP address Information ?

ARP

❖ Resolution Protocols for Name and Address Conversion

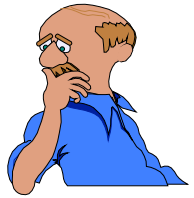
Name
(mmcn.ajou.ac.kr)

DNS
(IP-Name)

IP Address
(210.107.197.152)

ARP
(IP-MAC)

MAC Address
(00:50:da:89:92:84)



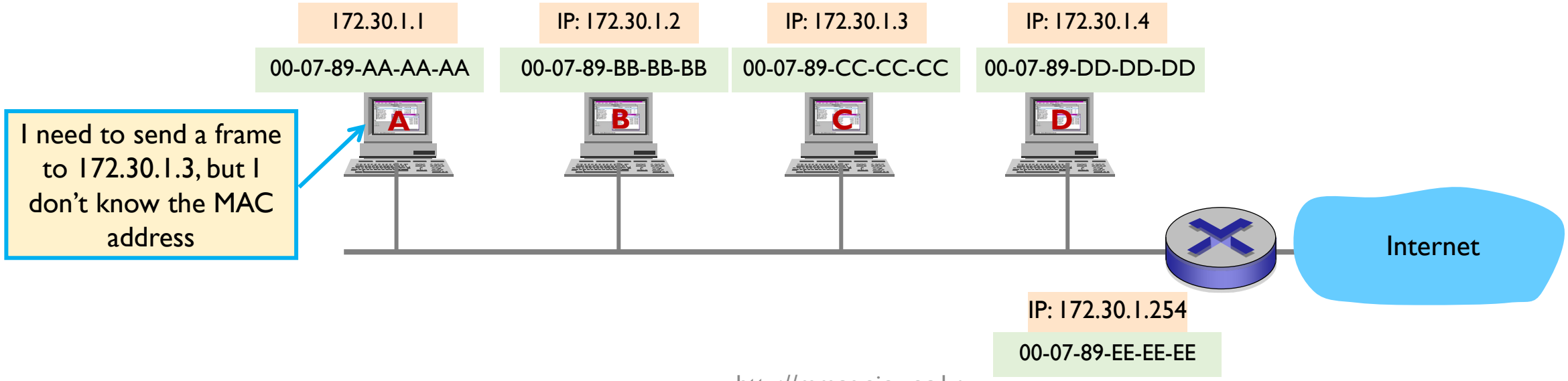
Application Layer

IP (Network) Layer

MAC (Data Link) Layer

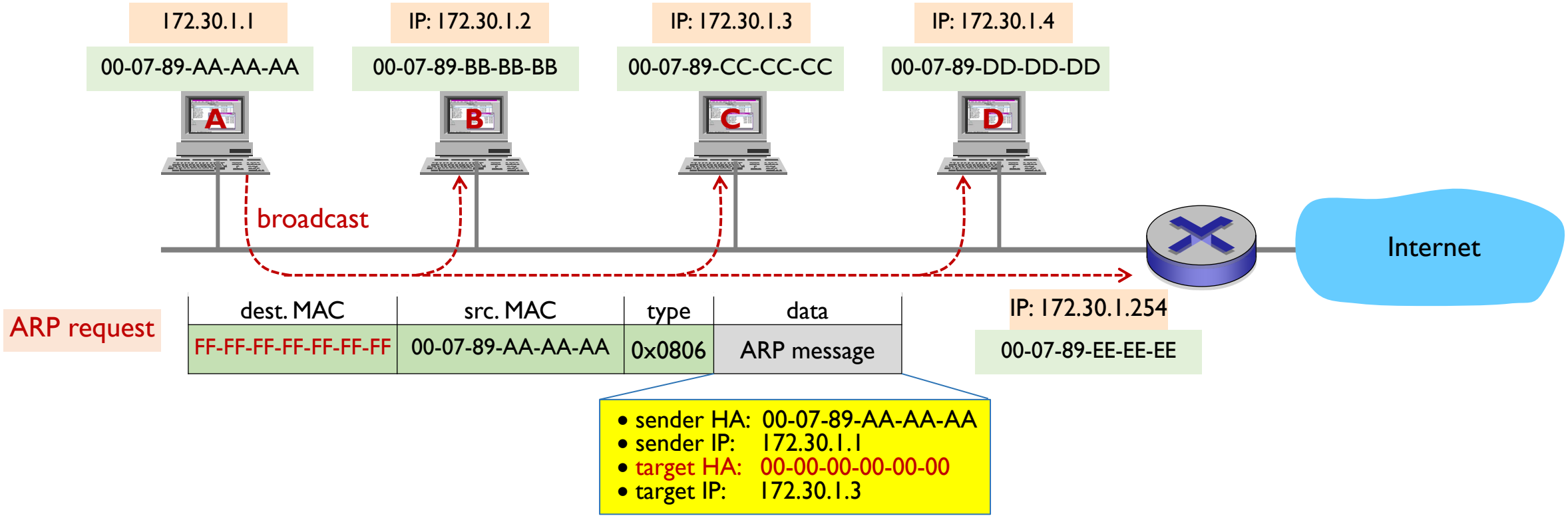
ARP Operation

- ARP Operation Environments
  - to send a frame on a LAN media,
    - hosts must know destination MAC address
  - host A wants to deliver a frame to host C
  - host A knows
    - only host C's IP address, but, host C's MAC address is not known)



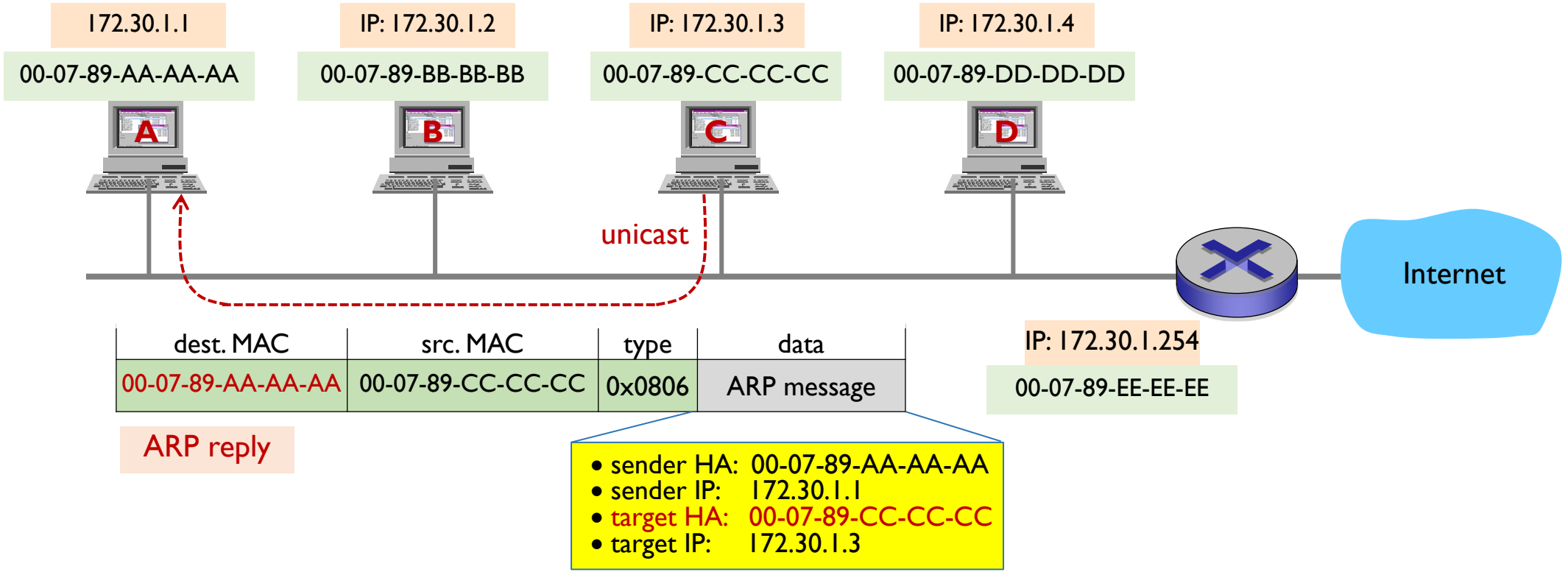
ARP Operation

- ARP Request
  - host A broadcasts ARP Request message to all hosts in its network,
    - in which host C's IP address is included



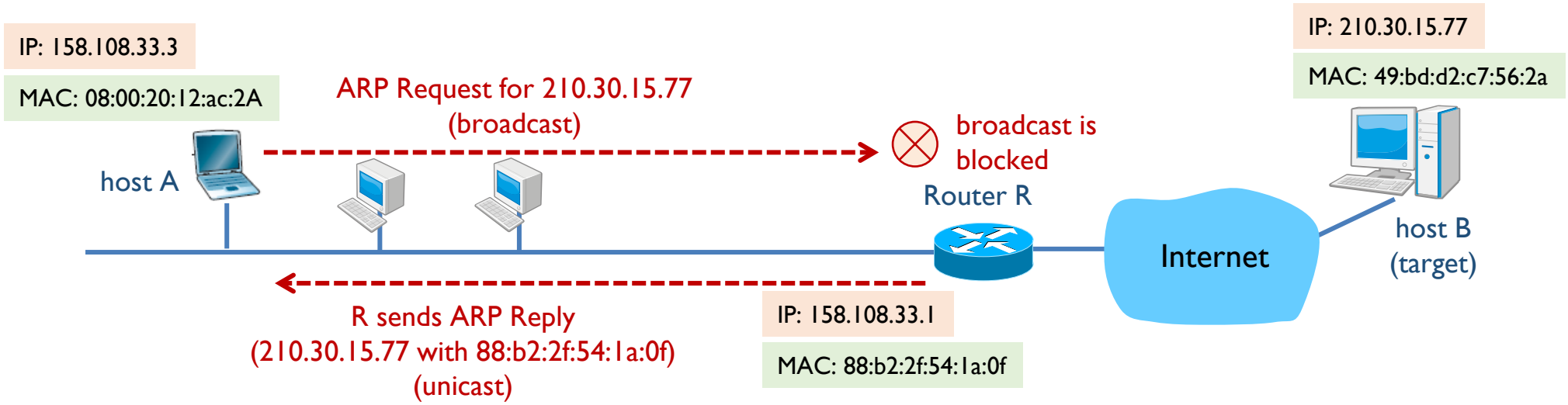
ARP Operation

- ARP Reply
  - host C – answers ARP reply message with its MAC address
  - host B, C, and D – keep host A's IP/MAC addresses in its ARP table for future use



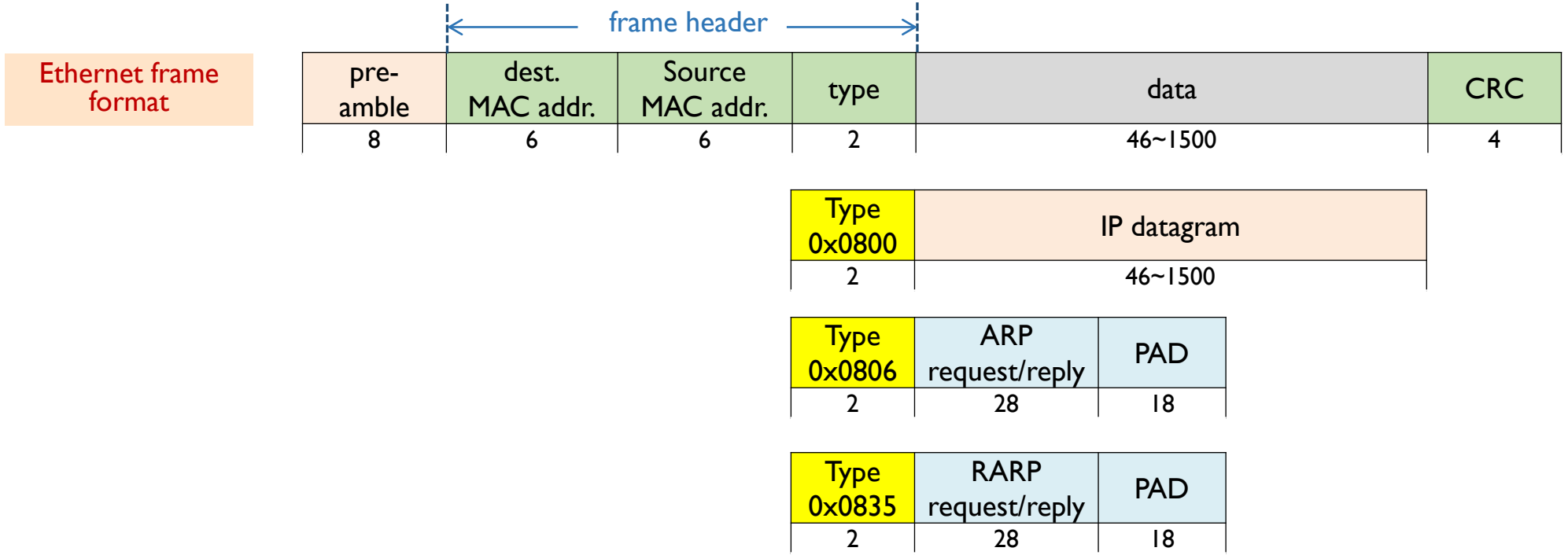
Proxy ARP

- Proxy ARP
  - Router blocks broadcasted ARP Request message
  - Proxy ARP allows
    - the router to respond for remote host outside the local network
    - example
      - host A wants to communicate with host B, but knows only B's IP address



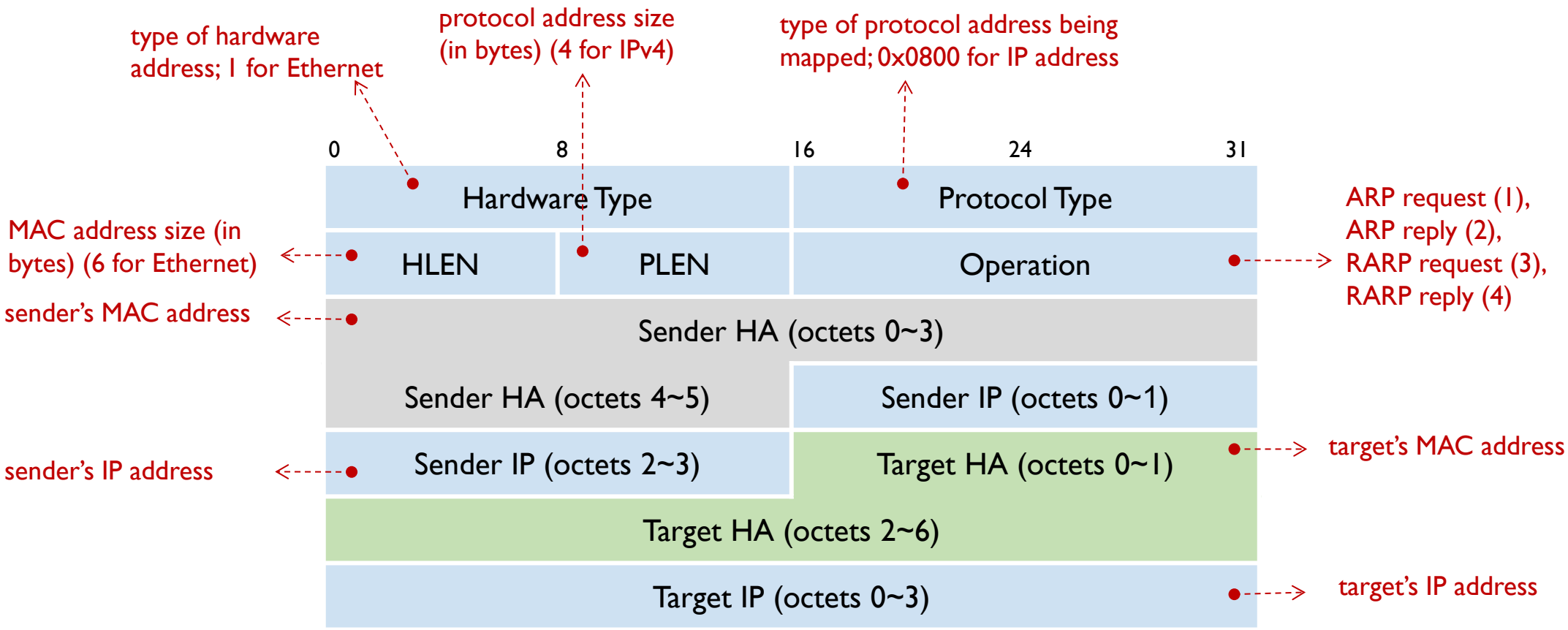
ARP Message Format

- Ethernet encapsulation
  - type field
    - 0x0800 : IP datagram
    - 0x0806 : ARP Request / Reply
    - 0x0835 : RARP Request / Reply



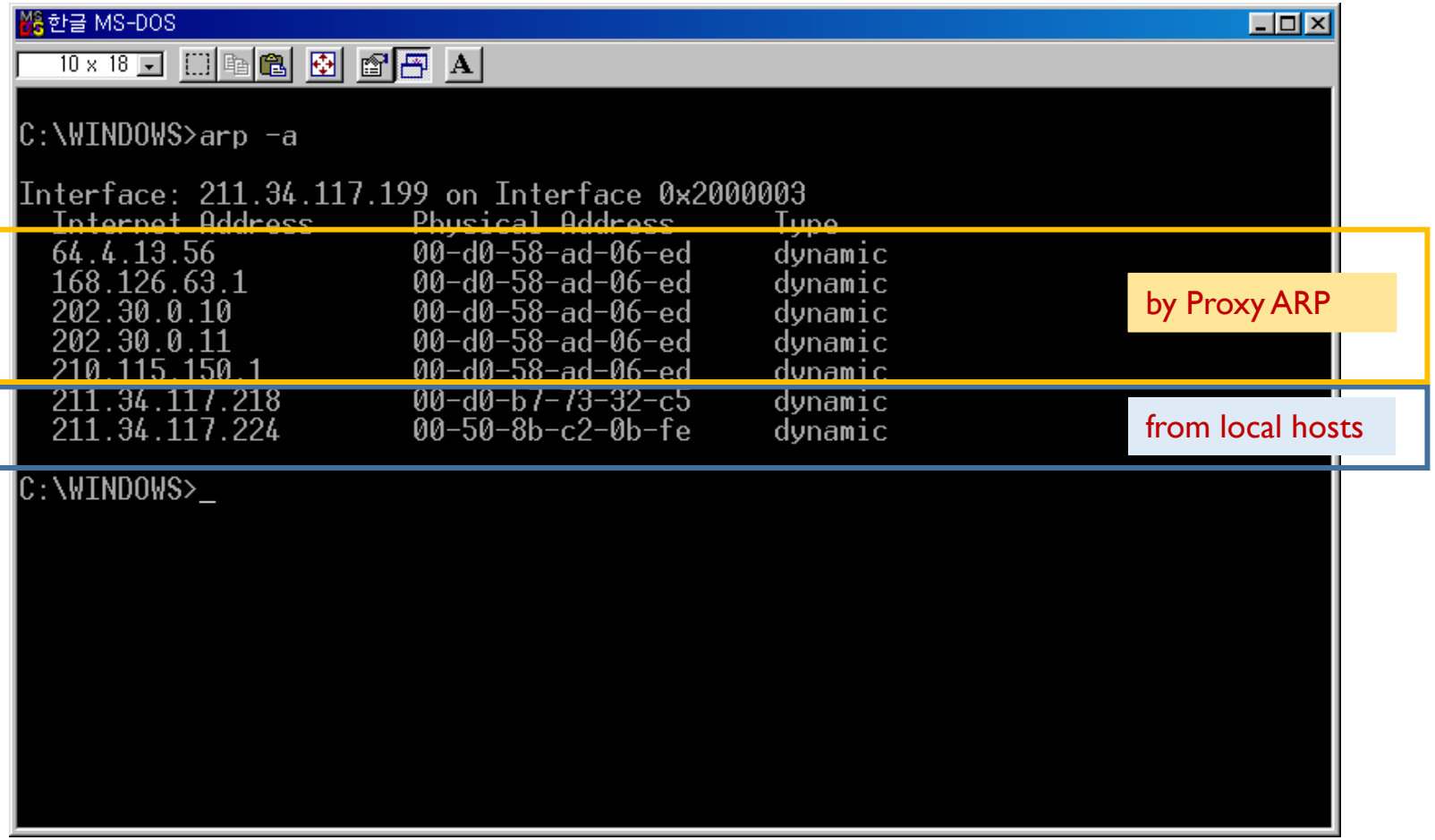
ARP Message Format

❖ ARP Message Format



ARP Caching

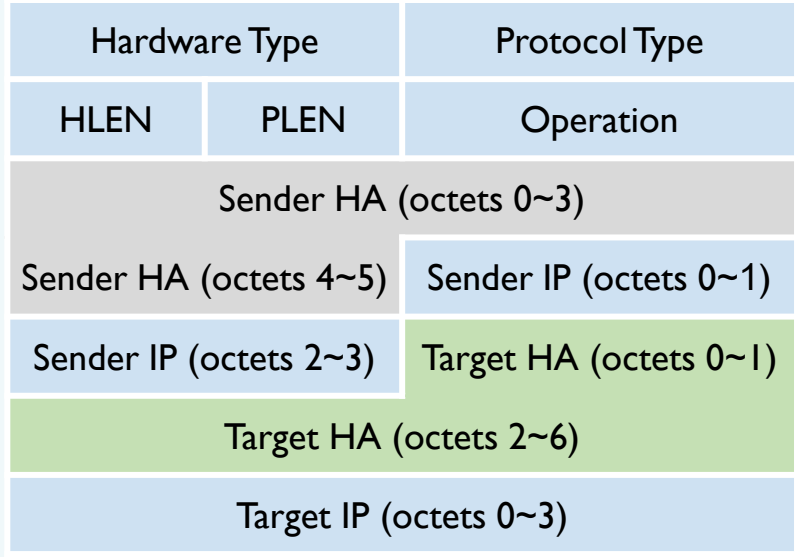
❖ ARP cache



Example ARP Generation

❖ Definitions and Header

```
#define       MAC_ADDR_LEN   6
#define       IPv4_ADDR_LEN  4
#define       ORIG_PACKET_LEN 64
#define       ARPOP_REQUEST  1              // ARP Request
#define       ARPOP_REPLY    2              // ARP Reply
#define       ARPHRD_ETHER   1              // Ethernet
#define       ETHERTYPE_IP   0x0800         // IP protocol
#define       ETHERTYPE_ARP  0x0806         // ARP protocol

struct arp_message
{
    uint16_t   ar_hrd;        // Format of hardware address
    uint16_t   ar_pro;        // Format of protocol address
    uint8_t    ar_hln;        // Length of hardware address
    uint8_t    ar_pln;        // Length of protocol address
    uint16_t   ar_op;         // ARP opcode (command)
    uint8_t    ar_sha[6];     // Sender hardware address
    uint32_t   ar_sip;        // Sender IP address
    uint8_t    ar_tha[6];     // Target hardware address
    uint32_t   ar_tip;        // Target IP address
}
```



Example ARP Generation

❖ ARP packet generation (1/2)

```
void   generate_packet_arp(u_char *packet, int op, u_char* smac, u_char* tmac, int *pklen)
{
    u_char                  sendbuf[4096] = { 0 };
    struct eth_header       *eth;
    struct arp_message      *arph;
    uint32_t                src_ip=0, dst_ip=0;

    /* Prepare headers*/
    eth       = (struct eth_header*)sendbuf;
    arph      = (struct arp_message*)(sendbuf + sizeof(struct eth_header));

    // Ethernet header
    etype = ETHERTYPE_ARP;                  // ethernet type for ARP
    get_header_ethernet(eth);

    // IP address manipulation
    inet_pton(AF_INET, sip, &src_ip);       // source IP address
    inet_pton(AF_INET, dip, &dst_ip);       // destination IP address
```

Example ARP Generation

- ARP packet generation (2/2)

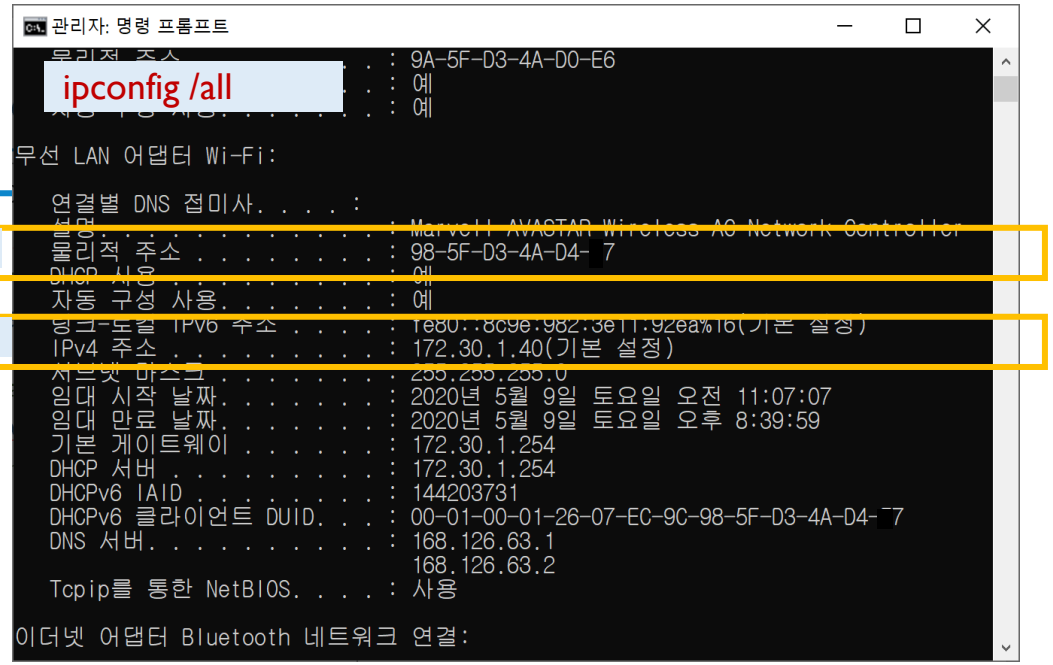
```
    // ARP Message
    arph->ar_hrd = htons(ARPHRD_ETHER);           // Format of hardware address
    arph->ar_pro = htons(ETHERTYPE_IP);           // Format of protocol address
    arph->ar_hln = MAC_ADDR_LEN;                  // Length of hardware address
    arph->ar_pln = IPv4_ADDR_LEN;                 // Length of protocol address
    arph->ar_op  = htons(op);                     // ARP opcode (1:request, 2:reply)

    memcpy(arph->ar_sha, smac, MAC_ADDR_LEN);     // Sender hardware address
    arph->ar_sip = src_ip;                        // Sender IP address
    memcpy(arph->ar_tha, tmac, MAC_ADDR_LEN);     // Target hardware address
    arph->ar_tip = dst_ip;                        // Target IP address

    *pklen = sizeof(struct eth_header) + sizeof(struct arp_message);
    if (*pklen <= ORIG_PACKET_LEN) *pklen = ORIG_PACKET_LEN;

    memcpy(packet, sendbuf, sizeof(sendbuf));
}
```

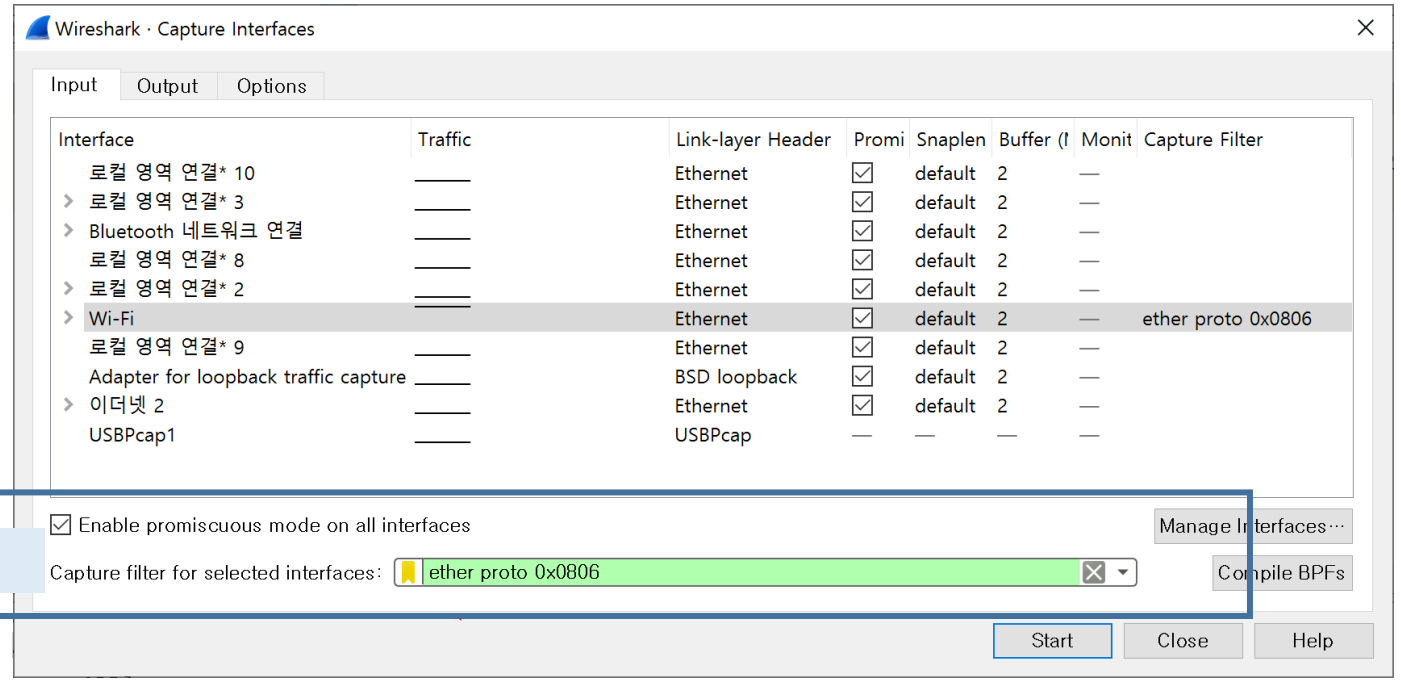
Example ARP Generation

```
uint8_t  dst_mac[6] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };          // Broadcast MAC
uint8_t  src_mac[6] = { 0x14, 0x18, 0xc3, 0x7e, 0xdc, 0x0e };          // Source MAC

int main()
{
    ...
    pcap_findalldevs(&alldevs, errbuf);    // Retrieve the device list
    ...
    adhandle = pcap_open_live(d->name, 65536, 1, 1000, effbuf);  // Open the adapter

    sip = "192.168.0.10";
    dip = “192.168.0.12";
    u_char    smac[6] = { 0x14, 0x18, 0xc3, 0x7e, 0xdc, 0x0e };  // sender mac address in ARP
    u_char    tmac[6] = { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };  // target mac address in ARP

    generate_packet_arp(packet, arp_op, smac, tmac, &pklen);     // ARP Packet generation

    pcap_sendpacket(adhandle, packet, pklen);      // Send packet
}
```

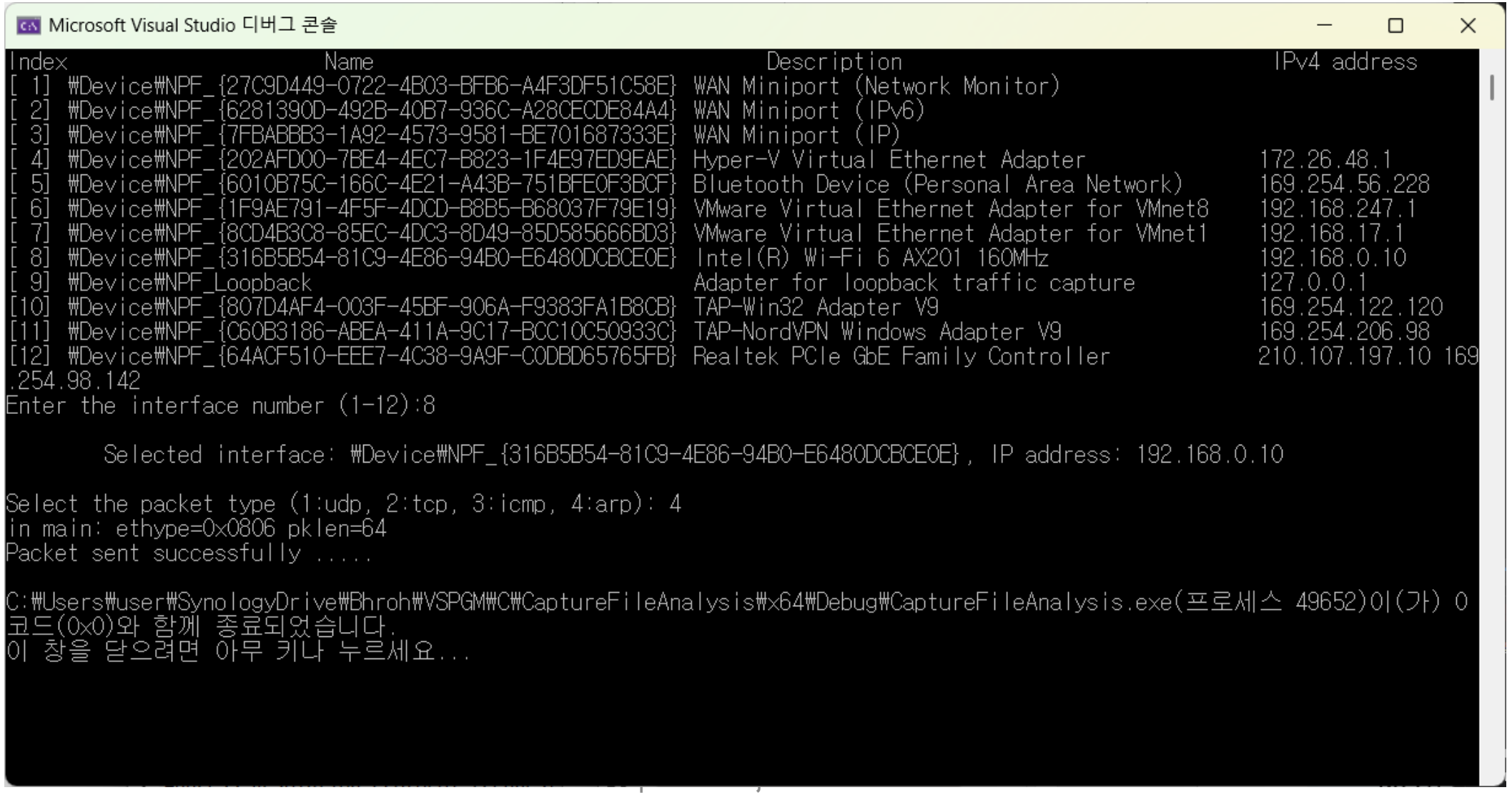
Example ARP Generation

- To run the example
  - check your ip and mac address
    - use “ipconfig /all” in command prompt
  - flush ARP cache
    - “arp –d” in command prompt (as administrator mode)
  - set filter in WireShark
    - Capture > Option
    - “ether proto 0x0806” (or arp)
  - then
    - capture start (WireShark)
    - run the program



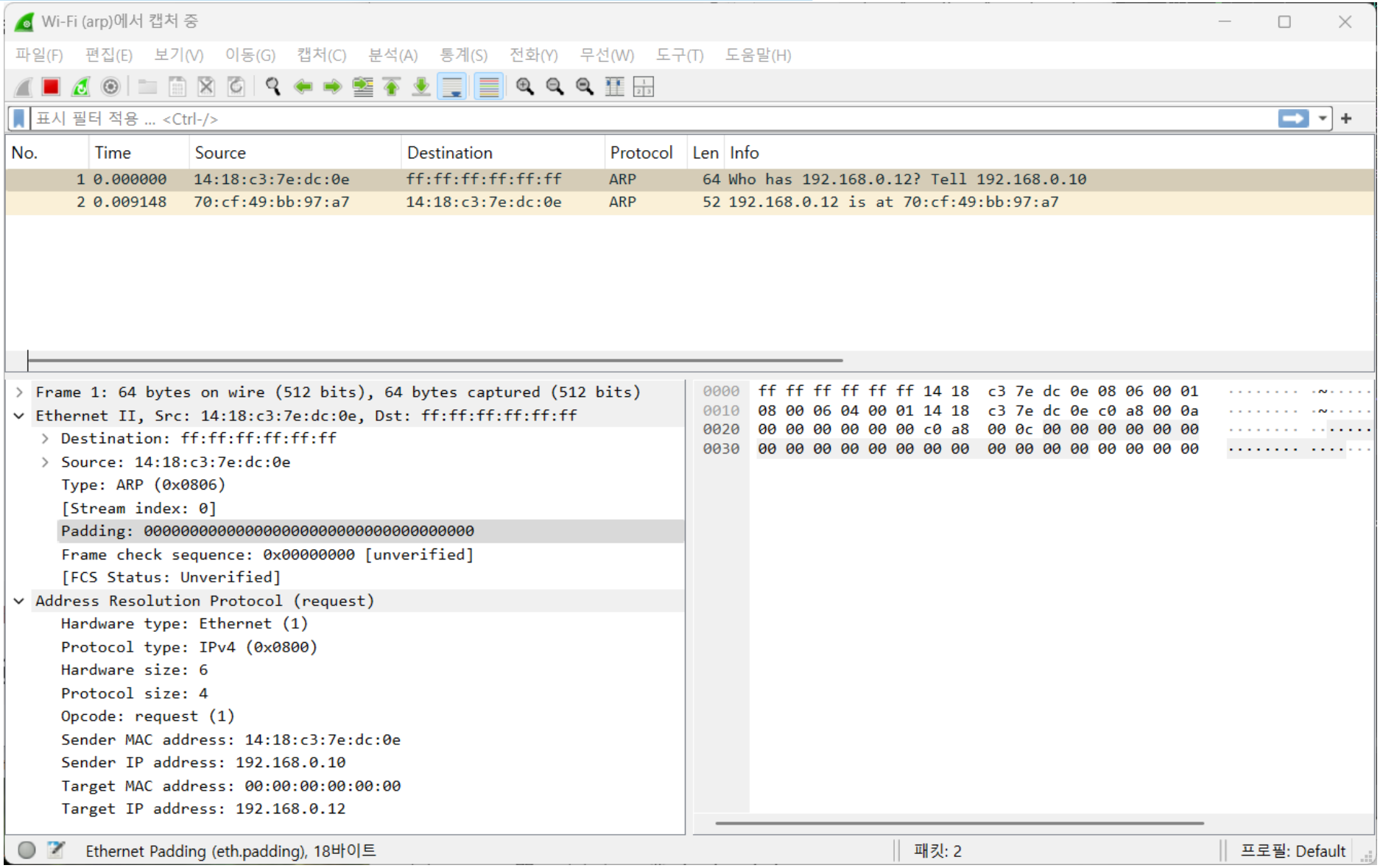


Example ARP Generation



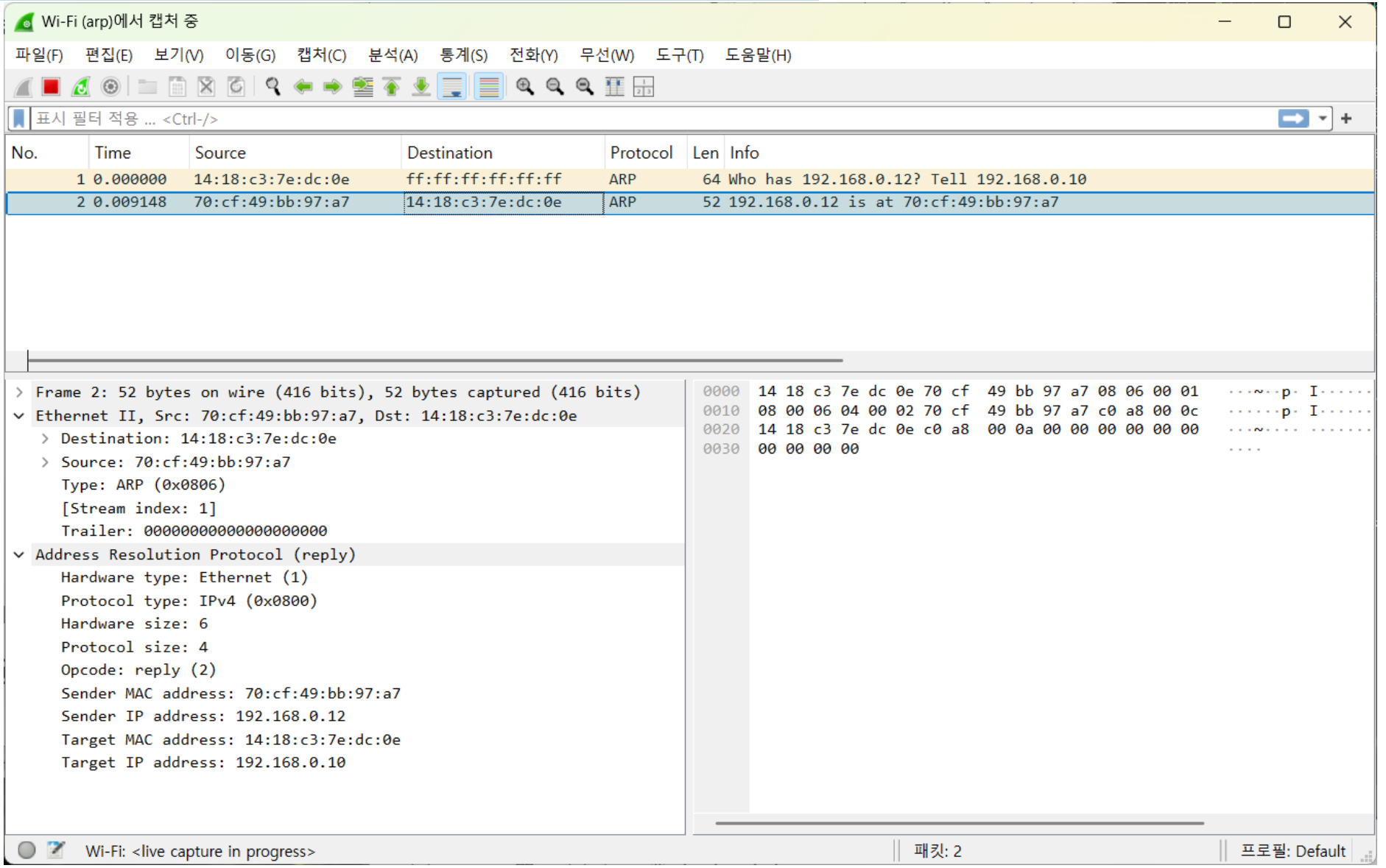
Example ARP Generation

- WireShark (1/2)



Example ARP Generation

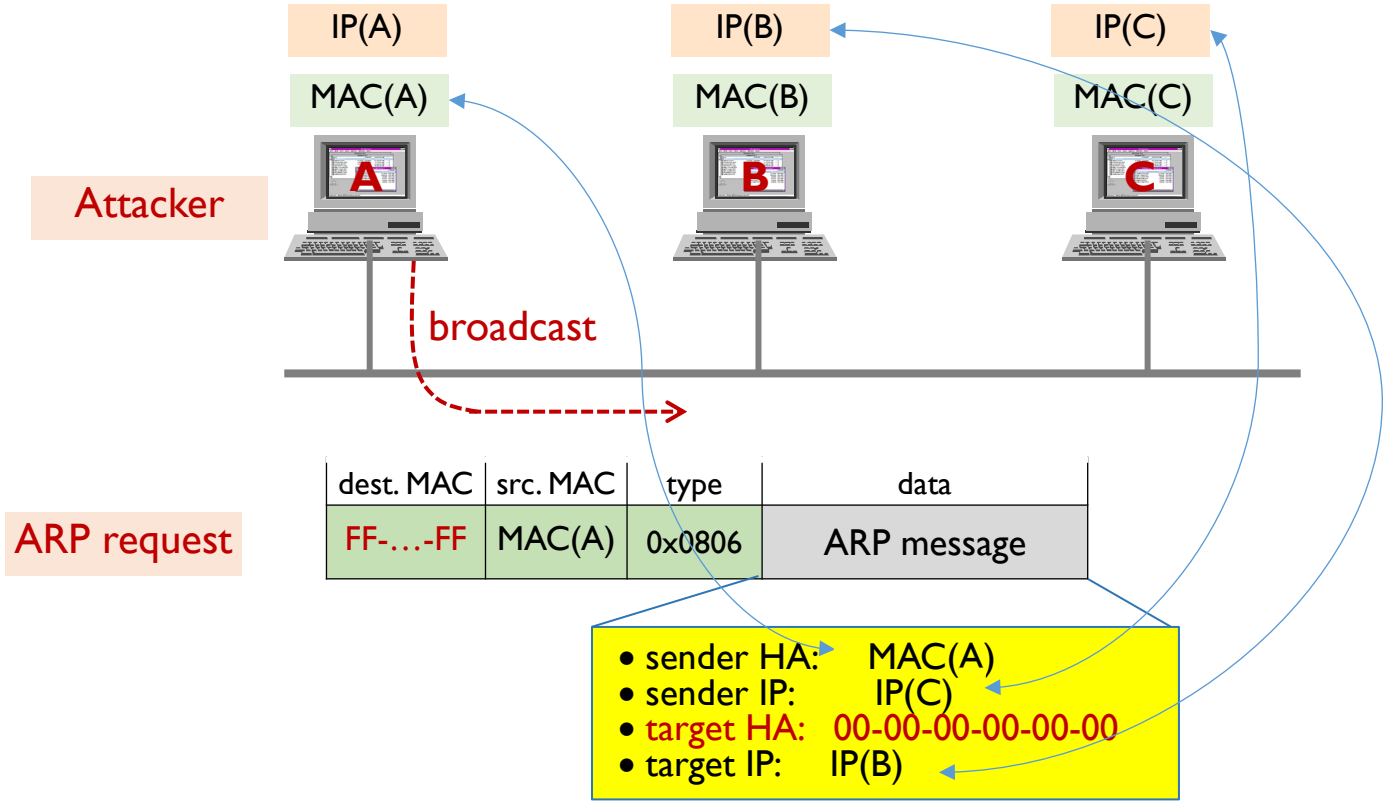
- WireShark (2/2)



PRACTICE – ARP POISONING

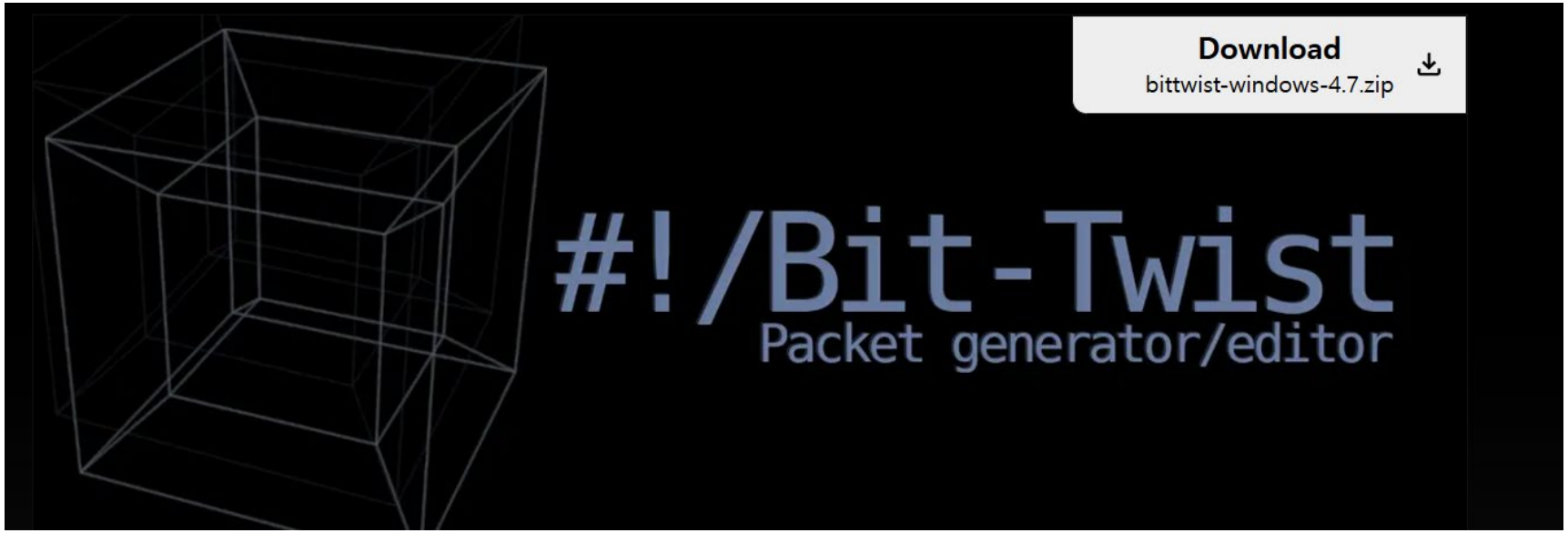
ARP Poisoning

❖ How does it work ?



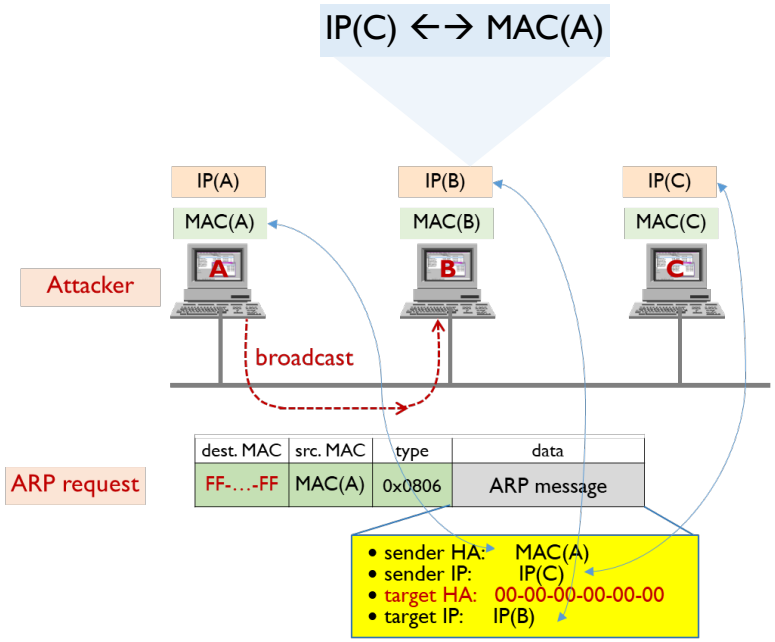
FYI

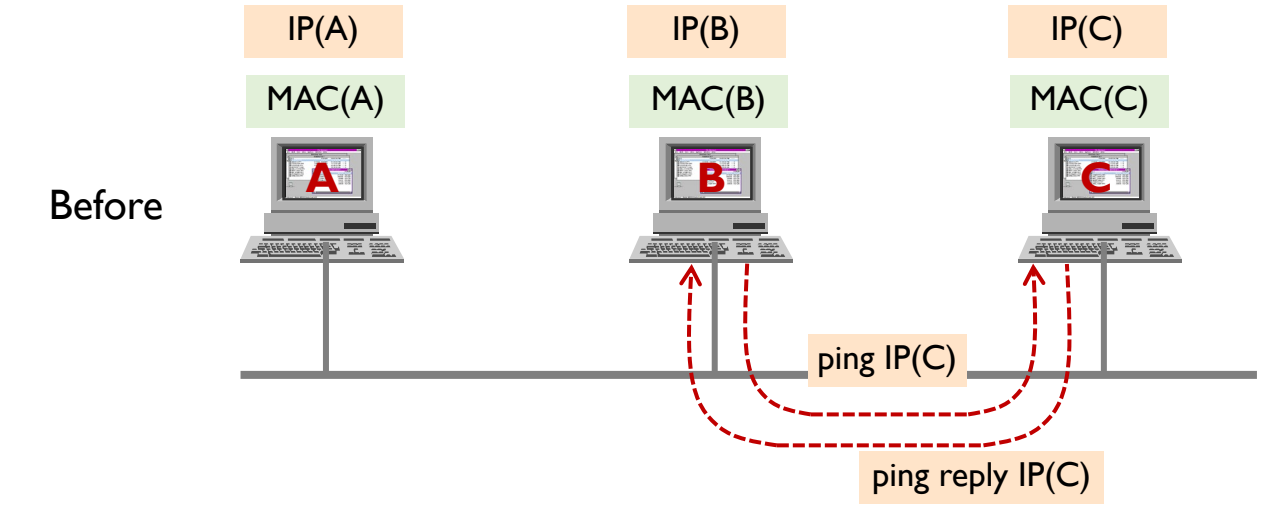
- Bit-Twist
  - https://bittwist.sourceforge.io/index.html

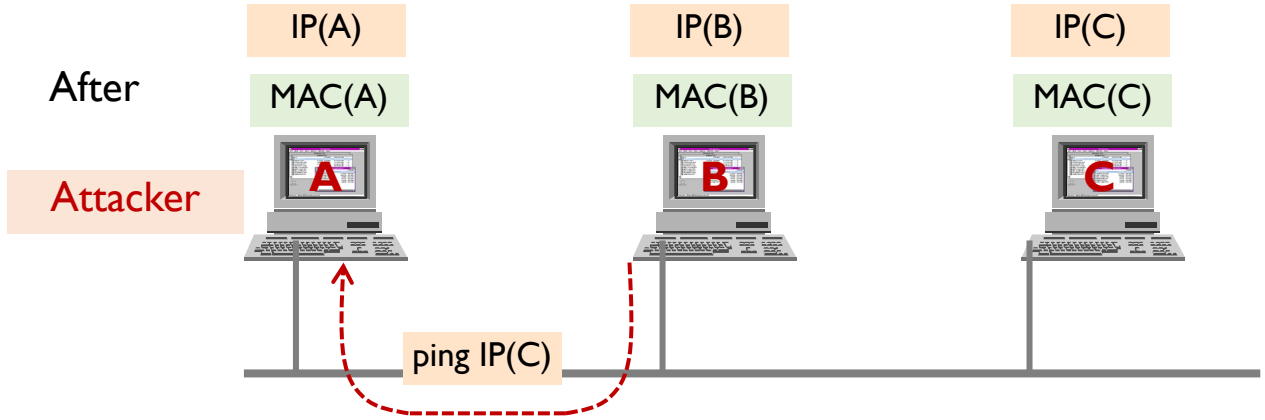


ARP Poisoning

❖ How does it work ?







Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN

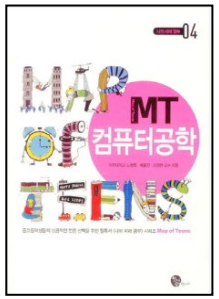
Mobile **Multimedia Convergence** Network Lab.

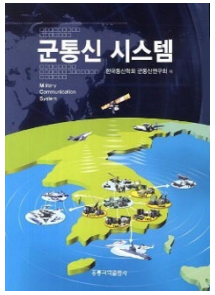
Homepage: http://mmcn.ajou.ac.kr

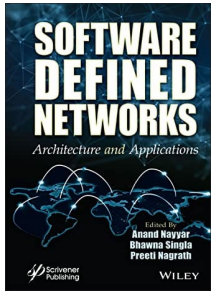
facebook: http://www.facebook.com/#!/groups/190702010947314

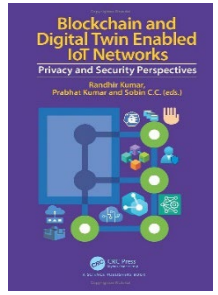
BK21 FOUR

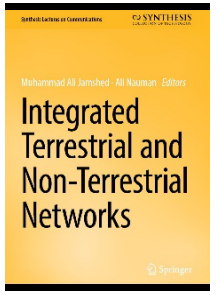
AI융합네트워크학과

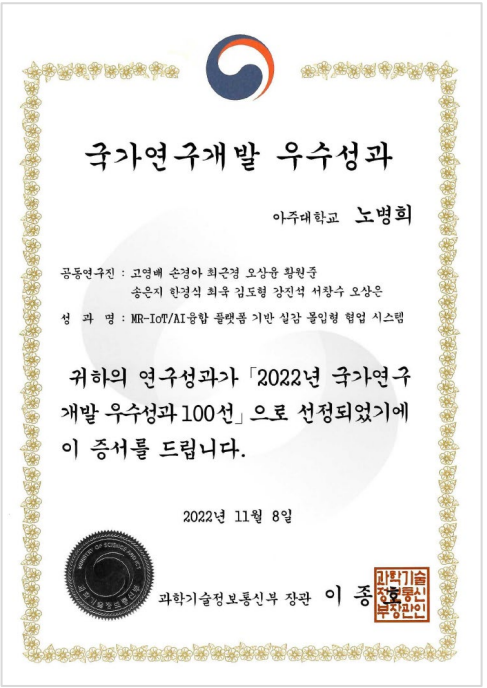












국가연구개발 우수성과

아주대학교 노병희

공동연구진 : 고영배 손경아 최준혁 오상윤 황원준
송은지 한경식 최욱 김도형 강진석 서창수 오상은

성 과 명 : MR-IoT/AI융합 플랫폼 기반 실감 몰입형 협업 시스템

귀하의 연구성과가 「2022년 국가연구개발 우수성과100선」으로 선정되었기에 이 증서를 드립니다.

2022년 11월 8일

과학기술정보통신부 장관 이 종 호